ABSTRACT

Background and Problem Statement : In the competitive landscape of social engineering, bridging the gap between the gap between academic knowledge and industry requirements is a significant challenge for students and job seekers. Traditional interview preparation platforms often lack personalization, providing generic question banks without feedback on soft skills or deep code analysis. Candidates frequently struggle to simulate the pressure and interactive nature of real-world technical interviews, leading to underperformance despite having strong theoretical knowledge.

Proposed Solution: This project presents “Interview Agent”, a full-stack web application designed to democratize access to high-quality, personalized interview coaching. The platform leverages Large Language Models (LLMs) to simulate the entire recruitment lifecycle, offering a holistic preparation environment that adapts to the user's specific career goals and experience level.

Key Methodology & Features The system is built upon a modern Microservices-inspired Architecture, utilizing Next.js for a responsive, high-performance frontend and FastAPI for an asynchronous, efficient backend. The core intelligence is powered by OpenAI's GPT models, integrated via a custom service layer (

llm.py) that handles prompt engineering and response parsing.

Key Modules Include Daily Assessment Engine For Curriculum Aligned Multiple Choice
Fresh questions each day, mixing Java,DSA, while weaving in OOPs ideas to build steady practi ce. Though focused on depth, they open space for thinking, not just answers. Because routine ma tters, these prompts return every morning, shaping small wins over time. One idea at a time, yet always linked - so gaps fill without force. Even when progress feels slow, repetition nudges gro wth quietly.
Intelligent coding arena with ai judge assessing more than just code output
What matters isn’t just whether it works. How fast it runs, how much memory it uses, how clean the structure is - those get examined too.
Feedback comes sharp and specific, not vague. Every suggestion points directly at improvement, skipping generic remarks. Standard tools often miss these layers
compilers cannot offer.

CHAPTER 01
INTRODUCTION

The Current Landscape of Technical Recruitment

In today's hyper-competitive technology sector, the recruitment process for software engineering roles has evolved into a rigorous, multi-stage gauntlet.

A typical route to getting ready often means jumping between scattered tools - tackling one-off challenges without clear links. Each piece stands apart, loosely connected at best. Learning h appens in pockets, rarely flowing from one area to the next. Some pieces fit together by accident rather than design. Gaps remain, unnoticed until they cause issues later on.

It's not always about ability - many students falter even when capable.

Frozen by the spotlight, even brilliant coders stumble on words. Speaking thoughts aloud trips up, despite sharp skills behind the screen.

On the flip side, someone could explain ideas clearly yet freeze up at a weird bug deep in code problem.

The Genesis of Interview Agent

Interview Agent was conceived to dismantle these barriers. It is not just another coding platform; it is a holistic AI-Powered Career Coach. The core philosophy behind Interview Agent is that interview preparation should be an immersive, integrated experience.

This project serves as a bridge between the academic curriculum of the Master of Computer Applications (MCA) program and the practical demands of the tech industry. It addresses the critical need for personalized, accessible, and realistic interview practice. Unlike human mentorship, which can be expensive and hard to schedule, Interview Agent is available 24/7, providing instant, data-driven feedback that helps users iterate and improve at their own pace.

Technological Innovation

At the heart of Interview Agent lies a sophisticated orchestration of modern technologies. The platform is built on a robust Full-Stack Architecture:

Frontend: Developed with Next.js 14, utilizing Server-Side Rendering (SSR) for optimal performance and SEO. The UI is crafted with Tailwind CSS and Framer Motion, ensuring a fluid, "app-like" experience that feels premium and responsive.

Backend: Powered by FastAPI (Python)

Artificial Intelligence: The "brain" of the agent utilizes state-of-the-art Large Language Models (LLMs).

Real-Time Communication: Integration of WebRTC and speech synthesis APIs enables the "Voice Mode," allowing users to talk to the AI just as they would to a human interviewer.

Impact and Vision

This project is a testament to the potential of AI in EdTech—shifting from passive learning to active, simulated practice. It empowers students to fail safely in a simulated environment so they can succeed confidently in the real world.

1.1. Project Description

1.1.1. Landing Page: The First Impression

The Landing Page is meticulously designed to convert curiosity into action.

Visual Language: Adopting a "Glassmorphism" design system, the page features translucent cards, soft gradients (deep blues and purples), and subtle background animations that evoke a sense of modern technology.

Hero Section: The centerpiece is a bold, animated headline: "Master Your Tech Interview." A call-to-action (CTA) button, "Start Practicing Free," pulses gently, drawing the eye.

Feature Showcase: A "Bento Grid" layout breaks down complex features into digestible visual bites. One card might show a snippet of code being analyzed, while another visualizes the voice wave of the AI interviewer.

Social Proof & Trust: Although a student project, the design includes placeholder sections for "Success Stories" and "Partner Companies" to simulate a production-ready SaaS environment.

1.1.2. Authentication (Login/Signup): Security Meets UX

The entry points to the application are the Login and Signup pages.

Secure Flow: Utilizing JWT (JSON Web Tokens), the authentication system is secure and persistent.

User Experience: The forms are clean, with real-time validation for email formats and password strength. A "Forgot Password" flow is included to handle account recovery.

Visuals: These pages often feature a split-screen layout—one half containing the form, the other displaying an inspiring graphic or a quote about career growth, keeping the user motivated even during administrative tasks.

1.1.3. The Dashboard: The Mission Control

The Dashboard is where the user spends most of their time. It is designed for "at-a-glance" comprehension.

Welcome Header: A personalized greeting ("Welcome back, [Name]!") coupled with a motivational quote of the day.

Stats Overview: Three key metrics dominate the top section: "Current Streak" (to encourage daily habit), "Problems Solved" (volume of practice), and "Global Rank" (gamification).

Action Cards: Large, clickable areas guide the user to the main modules. "Start Mock Interview" might feature a camera icon, while "Daily Coding Challenge" shows a code bracket symbol.

Progress Charts: A line graph visualizes the user's activity over the last 30 days, providing a visual reinforcement of their consistency.

1.1.4. Interactive Coding Arena: The Dojo

The Editor: Powered by Monaco Editor (the same engine as VS Code), it supports syntax highlighting, bracket matching, and auto-indentation for Python, Java, and C++.

Problem Pane: On the left, the problem description is presented with clear examples, constraints, and edge cases.

Test Case Visualization: Users can run their code against public test cases. The UI visually indicates "Pass" (Green) or "Fail" (Red) for each case, showing the expected vs. actual output.

Submission & Analysis: Upon submission, the backend evaluates the code against hidden test cases. The AI then generates a "Code Review," pointing out time complexity (Big O notation) and suggesting more idiomatic ways to write the solution.

1.1.5. AI Mock Interview Room: The Simulation

This is the flagship feature of the application.

Setup Phase: A "Green Room" checks the user's microphone and camera permissions, ensuring technical readiness.

The Interface: The screen is dominated by the AI Avatar (or a video placeholder) and the user's own video feed, mimicking a Zoom or Google Meet call.

Interaction: The AI speaks the questions. The user responds verbally. A "Listening" indicator shows when the AI is processing speech.

Feedback Report: Immediately after the session, a detailed report is generated. It scores the user on "Technical Accuracy," "Communication Clarity," and "Confidence," offering specific tips (e.g., "You said 'um' 15 times," or "You missed the edge case of negative numbers").

1.1.6. Assessment Centre: The Knowledge Check

The Assessment page offers structured quizzes to test theoretical knowledge.

Domain Selection: Users can filter quizzes by topic (e.g., DBMS, OS, Networking).

Adaptive Timer: Questions are timed to simulate the pressure of an online screening test.

Result Analytics: Post-quiz, users see a breakdown of their strong and weak areas. If they fail the "Pointers" section in C++, the system recommends specific resources to study.

1.1.7. User Profile: The Portfolio

The Profile page aggregates all data into a professional summary.

Resume Parsing: Users upload their PDF resume. The system parses it to extract skills and experience, which populates the profile fields automatically.

Skill Radar: A hexagonal radar chart visualizes the user's proficiency across 6 axes (e.g., Coding, System Design, Soft Skills).

Activity History: A scrollable list of every interview and problem attempted, allowing users to revisit past feedback.

1.2.2. The Development Journey

A few months into the term, the work began unfolding slowly. Over weeks, progress crept forward without hurry. Time passed while tasks layered one after another. By mid-semester, efforts had settled into a quiet rhythm. The stretch lasted until assignments reached their natural close

Starting out, it became clear the available options had limits. Some cost too much, like personal coaching. Others focused on just one thing - say, LeetCode testing code but ignoring how people explain their thinking. A gap showed up where help was broad enough yet affordable.

Starting off, a deep dive into what OpenAI's API can actually do. Testing how fast voice becomes text came next. Speed mattered, so Next.js entered the picture. On top of that, Python was chosen because it plays well with AI tools. Picking the correct mix of technologies shaped up step by step.

1.2.3. Challenges Overcome

Finding smooth progress proved tough at times.

What made chat flow like real talk? Trimming wait time between voice capture, brainy reply crafting, and spoken response. At first, silence stretched five seconds long. Tweak after tweak sliced that gap until replies snapped back almost instantly.

Fresh off the press, running untrusted code needs safeguards. Using containers like Docker carved out safe spaces where scripts execute without spreading. That isolation layer? It took work, but it holds strong.

Possibly the trickiest part? Keeping track of everything during a live interview - what's being recorded, past questions, sound levels. This needed more than basic React tricks. Instead, layered patterns stepped in to hold it all together.

1.2.4. Future Aspirations

Right now it lives in classrooms, yet the dream for Interview Agent stretches far beyond. What begins as a thesis could one day reshape how people prepare for tough conversations.

A single person might chat with another, later on. Software steps in - quietly tracking what gets said. Updates may let actual humans connect face to face. One system sits back, watching closely, typing nothing aloud.

A college might use this system to organize practice interviews through its career office. One learner, given solid tools and drive, can create powerful software solving actual peer challenges.

CHAPTER 02
LITERATURE SURVEY

2.1 Existing and Proposed System

2.1.1 Survey of Existing Systems

The domain of technical interview preparation has seen significant growth over the past decade, evolving from static textbooks to interactive online platforms.

2.1.1.1 Leetcode and Hacker Rank algorithm practice sites

What most coders follow as a common way to write programs.
Sure thing comes down to having loads of coding puzzles sorted by how tough they are and what idea they cover.
Inside your browser, a coding environment runs tests on what you write - checks happen without showing details. Hidden evaluations follow every attempt, working quietly behind the scenes.
Textonly setups miss the personal touch of real interviews. Not built to act like actual conversati ons between people.
A job talk isn't just about answers. Thinking out loud matters most. Unclear questions? That's n ormal. Work through them step by step. Talking clearly while figuring things out shows skill. Sil ence feels heavy - keep ideas moving. Hesitation is fine. Getting stuck? Say so, then shift gears. What counts is how you move forward when unsure. Explaining your path beats rushing to an an swer. Watchers care more about clarity than speed
effectively while coding. Soft skills aren't something LeetCode checks.
The response comes as either a simple yes or no. Or it shows numbers on speed and storage use. What's missing is insight into quality
Amentor might say the names used for values lack clarity. For instance, picking labels that better explain their purpose would help. One suggestion: choose words that show what the data does. I nstead of short forms, full descriptions work well. Clear terms make reading easier later on. Tho ughtful naming reduces confusion down the line. It's about making code speak plainly.

2.1.1.2 Pramp and Interviewing io practice interviews with peers

People tackle challenges through practice talks with others or trained partners who role-play interviews.
Features: Users are paired via video call to interview each other.
A practice session unfolds when someone reads out prepared prompts.
Questions come one after another, guided by written directions meant to keep things on track.
Bumpy results show up when peers don't bring the same level of skill to the conversation.

2.1.2 The Proposed System: Interview Agent

Interview Agent proposes a holistic, AI-driven approach that synthesizes the best elements of the above systems while addressing their shortcomings. It is designed to be an "always-available," consistent, and personalized career coach.

2.1.2.1 Integrated Ecosystem

Unlike LeetCode (code only) or Pramp (video only), Interview Agent combines Coding and Communication in a single platform.

The "Hybrid" Interview: The system can conduct interviews that require both verbal explanation and code implementation, mirroring the format of companies like Google and Meta.

Context Awareness: The system ingests the user's Resume and the specific Job Description. This allows it to ask targeted questions (e.g., "I see you used React in your last project; how did you handle state management?") rather than generic ones.

2.1.2.2 AI as the Interviewer

By using an AI Avatar with Voice capabilities:

Consistency: The AI provides a standardized level of difficulty and feedback quality, eliminating the variance found in peer interviews.

Availability: The system is available 24/7, allowing users to practice for 20 minutes before bed or on weekends without scheduling conflicts.

Psychological Safety: Users can practice without the fear of judgment from a human peer, allowing them to build confidence in a low-stakes environment before facing real interviewers.

2.1.2.3 Granular, Multi-Modal Feedback

The proposed system goes beyond "Pass/Fail."

Verbal Analysis: It analyzes the transcript of the user's speech to detect filler words ("um," "uh"), pacing issues, and sentiment (confidence vs. hesitation).

Technical Analysis: It evaluates code optimality (Time/Space complexity) and readability, acting as a senior code reviewer.

In summary, while existing systems excel in their specific niches, Interview Agent offers a unified, intelligent, and accessible solution that prepares candidates for the entirety of the interview experience.

2.2 Feasibility Study

2.2.1 Technical Feasibility

AI & NLP: The core intelligence is powered by OpenAI's GPT-4o (or similar models). The API is robust, well-documented, and capable of handling the complex prompts required for an interviewer persona.

Real-Time Communication: WebRTC technology, supported by all modern browsers, enables real-time video and audio capture without requiring external plugins. Libraries like react-webcam and browser native SpeechRecognition APIs make integration straightforward.

Scalability: The backend, built on FastAPI, is asynchronous by design, allowing it to handle multiple concurrent interview sessions (WebSocket connections) efficiently. The use of Docker ensures that the application can be deployed consistently across different environments (dev, staging, production).

2.2.2 Operational Feasibility

Operational feasibility evaluates how well the system solves the problem and how easy it is to use and maintain.

User Experience (UX): The application is designed with a "Student-First" mindset. The interface is intuitive, requiring no training to use. The "Dashboard" provides clear navigation, and the interview flow mimics standard video call apps (Zoom/Meet).

Accessibility: Being a web-based application, it requires no installation. It runs on any device with a browser and internet connection, making it accessible to students with varying hardware capabilities (laptops, tablets).

Maintenance: The modular architecture (separating Frontend, Backend, and AI services) means that updating one component (e.g., upgrading the AI model) does not break the entire system.

2.2.3 Economic Feasibility

Economic feasibility analyzes the cost-benefit ratio of the project.

Development Costs: As a student project, the labor cost is zero (sweat equity). The primary costs are infrastructure.

Hosting: Platforms like Vercel (Frontend) and Render/Railway (Backend) offer generous free tiers for hobby projects.

Database: Neon or Supabase provide free PostgreSQL instances.

AI Costs: OpenAI APIs are pay-as-you-go. For development and testing, the cost is negligible (often <$5/month). Even at scale, the cost per interview is significantly lower than hiring a human coach ($100+/hour).

Return on Investment (ROI):

For the User: The "cost" is free or a low subscription. The "return" is a high-paying job. The ROI is virtually infinite.

For the Developer: The project serves as a high-quality portfolio piece, demonstrating full-stack and AI competence, which directly enhances career prospects.

2.3 Tools and Technologies Used

2.3.1 Frontend Development

The frontend is the user's window into the application. We utilized a modern, component-based stack to ensure responsiveness and maintainability.

Next.js 14 (React Framework): Chosen for its hybrid rendering capabilities. It allows for Server-Side Rendering (SSR) of landing pages (for SEO and speed) and Client-Side Rendering (CSR) for the interactive dashboard. Its file-based routing simplifies navigation structure.

TypeScript: Adds static typing to JavaScript. This was crucial for a project of this complexity, catching errors at compile-time (e.g., mismatched API response types) rather than runtime, significantly reducing bugs.

Tailwind CSS: A utility-first CSS framework. It allowed for rapid UI development without leaving the HTML. It ensures design consistency (colors, spacing) and makes implementing "Dark Mode" trivial.

Framer Motion: A production-ready motion library for React. It powers the smooth page transitions, the "typing effect" of the AI, and the interactive graphs on the dashboard, adding a layer of polish that enhances user engagement.

Monaco Editor: The code editor component that powers VS Code. Integrating this gives users a familiar, professional-grade coding environment with syntax highlighting and auto-completion.

Lucide React: A library of clean, consistent SVG icons used throughout the UI for better visual communication.

2.3.2 Backend Development

The backend serves as the brain, orchestrating data flow, AI processing, and database management.

FastAPI (Python): A modern, high-performance web framework.

Why Python: It is the native language of AI.

Why FastAPI: It is one of the fastest Python frameworks available. Its support for asynchronous programming (async/await) is critical for handling long-running AI tasks (like generating interview feedback) without blocking the server.

PostgreSQL: An advanced, open-source object-relational database system. It stores user profiles, interview history, coding problems, and chat logs. Its reliability and support for complex queries make it ideal for structured data.

SQLAlchemy (ORM): An Object-Relational Mapper. It allows us to interact with the database using Python classes and objects instead of writing raw SQL queries. This improves code readability and security (preventing SQL injection).

Pydantic: Used for data validation. It ensures that the data sent between the frontend and backend matches the expected format, reducing data-related errors.

2.3.3 Artificial Intelligence & APIs

OpenAI API (GPT-4o): The core intelligence engine. It is used for:

Generating dynamic interview questions based on resume context.

Web Speech API: Browser-native APIs used for converting the user's spoken audio into text (Speech-to-Text) and synthesizing the AI's textual response into speech (Text-to-Speech).

2.3.4 DevOps and Tools

Git & GitHub: Used for version control. Branching strategies were used to develop features in isolation before merging them into the main codebase.

VS Code: The primary Integrated Development Environment (IDE), enhanced with extensions for Python, React, and Docker.

2.4 Hardware and Software Requirements

2.4.1 Software Requirements

These are the prerequisites required to develop, deploy, and run the application.

For Development (Developer's Machine)

Operating System: Windows 10/11, macOS (Ventura or later), or Linux (Ubuntu 20.04+).

Runtime Environments:

Node.js: Version 18.x or higher (for Frontend).

Python: Version 3.10 or higher (for Backend).

Database: PostgreSQL 14 or higher (local installation or cloud instance).

IDE: Visual Studio Code (recommended) or PyCharm/WebStorm.

Version Control: Git 2.x.

Containerization: Docker Desktop (optional but recommended).

For Deployment (Server)

Web Server: Nginx or Apache (as a reverse proxy).

Process Manager: PM2 (for Node.js) and Gunicorn/Uvicorn (for Python).

Cloud Platform: AWS, Google Cloud, Vercel, or Render.

For the End User (Client)

Operating System: Any OS capable of running a modern web browser (Windows, macOS, Android, iOS).

2.4.2 Hardware Requirements

These are the physical resources needed.

For Development

Processor: Intel Core i5/Ryzen 5 (Quad-core) or better.

RAM: Minimum 8GB (16GB recommended for running Docker and multiple servers simultaneously).

Storage: 256GB SSD (Fast I/O is important for compilation).

Internet: Stable broadband connection for downloading packages and API access.

For the End User

Device: Laptop, Desktop, or Tablet.

CHAPTER 03
SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Users

The Interview Agent system is designed to cater to two primary categories of people privileges, and interaction patterns. Understanding these user personas defining the system's behavior and security constraints.

3.1.1 The Candidate (Primary User)

The Candidate is the core end-user of the application. This persona represents students, job seekers, and software engineers looking to improve their technical interview skills. The entire frontend experience is optimized for this user group.

Characteristics:

Goal: To practice coding problems, simulate interview scenarios, and receive actionable feedback to improve their chances of getting hired.

Frequency of Use: High. Candidates are expected to use the platform daily or weekly during their interview preparation phase.

Pain Points:

Lack of structured feedback in self-study.

Anxiety during real interviews due to lack of realistic practice.

Difficulty in identifying specific weak areas (e.g., "I know I'm bad at graphs, but why?").

Responsibilities & Privileges:

Account Management:

Registration: Create a secure account using email/password or OAuth (Google/GitHub).

Profile Customization: Update personal details, including target roles (e.g., "Frontend Developer", "Data Scientist") and experience level.

Resume Management: Upload, view, and replace PDF resumes. The user must ensure the resume is legible for the AI parser.

Practice & Simulation:

Coding Arena: Access the full library of coding problems. Users can filter by difficulty (Easy, Medium, Hard) and topic (Arrays, Trees, DP).

Mock Interviews: Initiate AI-driven video interviews. Users can choose the focus area (Behavioral, Technical, System Design) or let the AI decide based on their profile.

Assessments: Take timed quizzes to benchmark theoretical knowledge against other users.

Data Access & Analytics:

Performance Dashboard: View detailed analytics, including "Win Rate," "Average Time per Problem," and "Skill Radar" charts.

History Log: Access transcripts and audio recordings of past interviews for self-review.

Privacy Constraint: Candidates data, they strictly cannot view the profiles, code submissions, or interview logs of other candidates.

3.2 Functional Requirements

The specific behaviors and functions of the system—what the system does.

3.2.1 Authentication and Authorization Module

FR-01 User Registration: Sign up by providing a valid email address, a full name, and a secure password (min 8 chars, mixed case).

FR-02 User Login: The system shall authenticate users using their email and password. Upon successful login, the system shall issue a JSON Web Token (JWT) with a configurable expiration time (e.g., 24 hours).

FR-03 Password Encryption: The system shall store passwords in a hashed format using a strong algorithm (e.g., bcrypt with a work factor of at least 12) and never in plain text.

FR-04 Session Expiry: The system shall automatically invalidate the session token and log users out after a predefined period of inactivity to ensure security.

FR-05 Password Reset: The system shall provide a "Forgot Password" flow where a secure reset link is emailed to the user.

3.2.2 Dashboard and Profile Module

FR-06 Statistics Display: The dashboard shall display real-time metrics, including the user's current login streak, total problems solved, and an aggregate "Readiness Score" calculated from their interview performance.

FR-07 Resume Parsing: Upload a PDF resume. The backend shall parse this file to extract key skills (e.g., "Python", "React") and store them in the user's profile context for the AI to use.

FR-08 Progress Visualization: The system shall visualize the user's activity over the last 30 days using a contribution graph (similar to GitHub) or a line chart.

FR-09 Profile Editing: Ppdate their display name, bio, and preferred programming languages

3.2.3 AI Mock Interview Module

FR-15 AV Setup (Green Room): Provide a pre-interview interface to test and select their microphone and camera inputs. It should visualize audio levels to ensure the mic is working.

FR-16 Contextual Question Generation: The system shall generate interview questions dynamically based on the user's uploaded resume and the selected job role. For example, if the resume mentions "AWS", the AI should ask cloud-related questions.

FR-17 Real-Time Speech-to-Text: The system shall transcribe the user's spoken response into text in real-time using the Web Speech API or a cloud service.

FR-18 Text-to-Speech Synthesis: The system shall synthesize the AI's textual questions into natural-sounding audio, playing it back to the user to simulate a voice conversation.

FR-19 Interview State Management: The system shall maintain the state of the interview (Introduction -> Technical -> Behavioral -> Conclusion), ensuring a logical flow. The AI should not repeat questions.

3.2.4 Feedback and Reporting Module

FR-21 Report Generation: Immediately after an interview, generate a comprehensive feedback report.

FR-22 Scoring Algorithm: Assign scores (0-100) for "Technical Accuracy," "Communication Clarity," and "Confidence" based on the transcript analysis.

FR-23 Detailed Feedback: The report shall include specific actionable feedback, such as "You used filler words 15 times" or "Your explanation of HashMaps was vague."

FR-24 History Access: View reports from all previous interview sessions, allowing them to track their improvement over time.

3.3 Non-Functional Requirements

The system—how the system performs rather than what it does.

3.3.1 Performance Requirements

NFR-01 AI Latency: The AI's response time (from the moment the user stops speaking to the moment the AI starts speaking) should be under 2 seconds to maintain the illusion of a natural conversation. High latency breaks immersion.

NFR-02 Code Execution Speed: The Coding Arena must execute and return results for standard algorithms within 3 seconds (excluding network latency) to prevent user frustration.

NFR-03 Page Load Time: The critical rendering path (Dashboard, Landing Page) must load within 1.5 seconds on a standard 4G network (First Contentful Paint).

NFR-04 Database Query Performance: Common queries (e.g., fetching user profile, listing problems) must execute in under 100ms.

3.3.2 Scalability and Reliability

NFR-05 Concurrency: The backend system shall support at least 50 concurrent interview sessions without degradation in audio quality or API response times.

NFR-06 Availability: The system should aim for 99.9% uptime during business hours. Scheduled maintenance should be performed during off-peak hours.

NFR-07 Database Scalability: The database schema should be normalized to support the growth of user logs and coding submissions (up to 1 million records) without significant query performance penalties.

NFR-08 Horizontal Scaling: The application architecture should allow for horizontal scaling (adding more server instances) to handle increased load.

3.3.3 Security Requirements

NFR-09 Data Transmission: All data transmitted between the client and server (including video streams and API calls) must be encrypted using TLS 1.2/1.3 (HTTPS).

NFR-10 Code Sandboxing: User-submitted code must be executed in strictly isolated containers (e.g., Docker) with resource limits (CPU, RAM, Network) to prevent malicious attacks (e.g., fork bombs, infinite loops, reverse shells) from compromising the host server.

NFR-11 API Security: All API endpoints (except Login/Signup) must be protected via JWT Authentication. The system must validate the token signature on every request.

NFR-12 Input Validation: All user inputs (form data, code submissions) must be sanitized to prevent SQL Injection and Cross-Site Scripting (XSS) attacks.

3.3.4 Maintainability and Portability

NFR-13 Modular Codebase: The frontend and backend must be decoupled. Changes to the UI should not require a redeployment of the API server, and vice versa.

NFR-14 Documentation: The API must be documented using OpenAPI/Swagger standards to facilitate future development and mobile app integration.

CHAPTER 04
SYSTEM DESIGN

4.1 System Perspective

4.1.1 Introduction

The System Perspective defines the position of the Interview Agent within the broader technological environment. It delineates the boundaries of the system, identifying which components are internal to the application and which are external entities or services that the system interacts with. This perspective is crucial for understanding the system's dependencies, interfaces, and overall architectural style.

4.1.2 Architectural Style: Modular Monolith

The Interview Agent is architected as a Modular Monolith.

Definition: In this style, the entire backend application runs as a single deployable unit (a single process), but the internal code is organized into distinct, loosely coupled modules (e.g., Auth, Assessment, Coding, Interview).

Rationale:

Simplicity: Unlike a microservices architecture, a monolith is easier to develop, test, and deploy for a small to medium-sized team (or a single developer). There is no need to manage complex inter-service communication or distributed transactions.

Performance: Internal function calls are faster than network calls between microservices.

Modularity: By keeping modules separate (e.g., app/api/coding, app/api/interview), we retain the ability to split the application into microservices in the future if a specific module (like the resource-intensive AI Interviewer) requires independent scaling.

4.1.3 System Interfaces

The system interacts with several external entities and environments.

1. User Interfaces (Frontend)

Web Client: The primary interface is a Responsive Web Application built with Next.js. It runs in the user's browser.

Communication: It communicates with the backend via RESTful APIs (over HTTPS) for standard data operations (login, fetching profiles) and WebSockets for real-time features (live interview audio streaming).

Responsibilities: Rendering the UI, capturing user input (text, code, audio, video), and presenting feedback.

2. External Service Interfaces

The Interview Agent is not an island; it relies on powerful third-party APIs to deliver its core value.

OpenAI API (The Intelligence Layer):

Role: Acts as the "Brain" of the interviewer.

Data Flow: The backend sends prompts (containing user context and resume data) to OpenAI and receives generated text (questions, feedback, code analysis).

Protocol: HTTPS (JSON REST API).

Database Service (PostgreSQL):

Role: Persistent storage for all structured data.

Data Flow: The backend reads/writes user profiles, authentication tokens, problem definitions, and interview logs.
Protocol: TCP/IP (PostgreSQL Wire Protocol).
Speech Services (Web Speech API / Cloud STT):
Role: Converts audio to text and vice versa.
Data Flow: Audio streams are processed to generate transcripts for the AI to analyze.

4.1.4 Operating Environment

Server-Side: The backend is designed to run in a Linux environment (e.g., Ubuntu within a Docker container). It requires a Python runtime (3.10+) and access to the internet to reach external APIs.
Client-Side: The application is platform-agnostic. It runs on any device with a modern web browser (Chrome, Firefox, Edge, Safari) that supports HTML5, WebRTC (for media), and JavaScript (ES6+).

4.1.5 Design Constraints

Network Dependency: The system is heavily dependent on a stable internet connection. The AI features (OpenAI) and real-time interviews cannot function offline.
Latency Sensitivity: The "Voice Mode" requires low latency. The system architecture must minimize the number of hops between the user and the AI processing service to keep the conversation natural.
Security Compliance: Since the system handles user resumes and potentially personal voice data, it must adhere to basic data privacy principles (e.g., not storing raw audio longer than necessary, encrypting data at rest).

4.2 Context Diagram (Level 0 DFD)

4.2.1 Overview

The Context Diagram (also known as the Level 0 Data Flow Diagram) provides the highest-level view of the system. It treats the entire Interview Agent application as a single "Black Box" and focuses solely on the flow of information between the system and external entities. It answers the question: Who interacts with the system, and what data do they exchange?

4.2.2 Diagram Representation

1. Registration Details (Email, Password)
2. Resume (PDF)
3. Code Submission
4. Audio/Video Input
5. Auth Token (JWT)
6. Interview Feedback & Score
7. Code Verdict (Pass/Fail)
8. AI Voice Response
9. New Coding Problems
10. User Ban/Moderation
11. System Usage Reports
12. Prompt + Context
13. Generated Question/Feedback
14. Read/Write Data
15. User Administrator Interview Agent System OpenAI API Service PostgreSQL Database

4.2.3 Detailed Data Flow Description

Entity 1: The Candidate (User)

The Candidate is the primary source of input and the primary consumer of output.

Input Flows (User -> System):

Registration Data: PII (Personally Identifiable Information) like Name and Email, along with security credentials (Password).

Resume: The user uploads a document. This is a complex data object that the system must parse.

Code Submission: During a coding challenge, the user sends raw source code (text strings) to the system for evaluation.

Audio/Video Stream: During an interview, continuous streams of audio and video data are sent to the system.

Output Flows (System -> User):

Authentication Token: Upon login, the system issues a secure token (JWT) that the user's browser must store and send with future requests.

Feedback & Reports: After an interview, the system generates a structured report (JSON data rendered as HTML) containing scores and text feedback.

AI Voice Response: The system streams synthesized audio back to the user, creating the "voice" of the interviewer.

Entity 2: The Administrator

The Administrator manages the system's content and health.

Input Flows (Admin -> System):

Problem Definitions: Admins input new coding problems, including descriptions, constraints, and hidden test cases.

Moderation Commands: Actions to ban users or delete inappropriate content.

Output Flows (System -> Admin):

Analytics: The system aggregates data (e.g., "100 new users today") and presents it to the admin.

Entity 3: OpenAI API Service

This external entity provides the intelligence. The system acts as a client to this service.

Outbound Flow (System -> OpenAI):

Prompts: The system constructs complex text prompts. Example: "You are a Senior Java Developer. Interview a candidate who has 2 years of experience. Ask a question about Garbage Collection."

Inbound Flow (OpenAI -> System):

Completions: The API returns the text response. Example: "Can you explain how the Mark-and-Sweep algorithm works?"

Entity 4: PostgreSQL Database

While technically part of the backend infrastructure, in a context diagram, the database is often visualized to show where data "rests."

Flows: The system sends SQL Queries (INSERT, SELECT, UPDATE) and receives Result Sets (Rows of data). This flow represents the persistence of the system's state.

4.2.4 Boundary Definition

The boundary of the Interview Agent includes:

The Frontend Application (running in the browser).

The Backend API Server (running in the cloud/server).

The Code Execution Sandbox (Docker containers).

Everything else—the user's microphone hardware, the OpenAI servers, the email delivery network—is considered external to the system. This clear boundary definition helps in scope management, ensuring the development team focuses only on what they can control.

CHAPTER 05
DETAILED DESIGN

5.1 Use Case Diagram

5.1.1 Overview

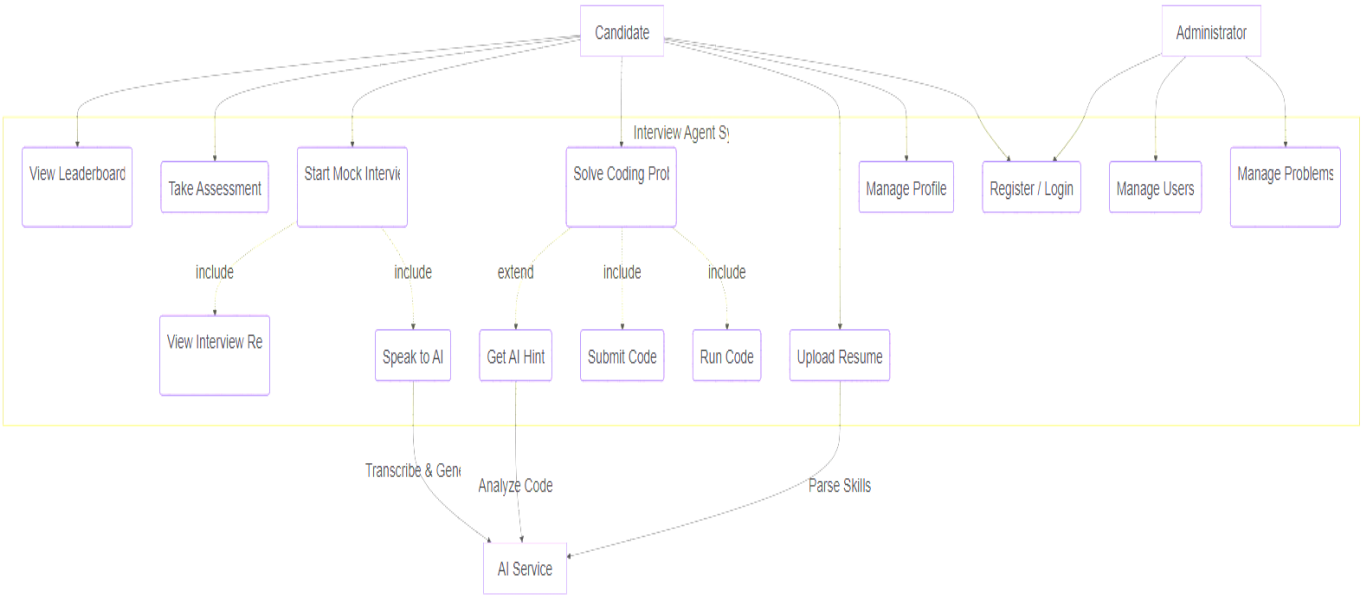
This captures the dynamic behavior of the Interview Agent system. It identifies the primary actors (users and external systems) and their interactions with the system's functionalities (use cases).

5.1.2 Actors

1. Candidate (Primary Actor): The end-user who uses the system to practice coding and take mock interviews.
2. Administrator (Secondary Actor): The system owner who manages content (problems, questions) and users.
3. AI Service (External Actor): The OpenAI API that provides intelligence for interviews and code analysis.
4. Database (System Actor): The storage engine responsible for persisting user data.

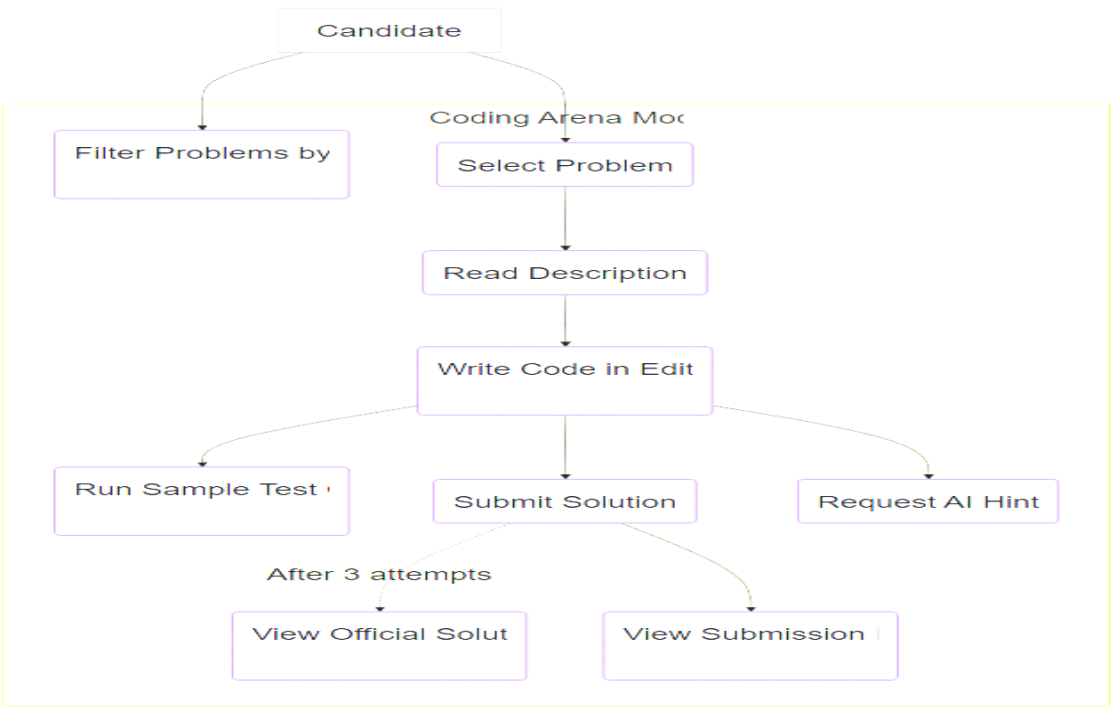
5.1.3 System-Level

It provides a high-level view of the entire system's functionality.



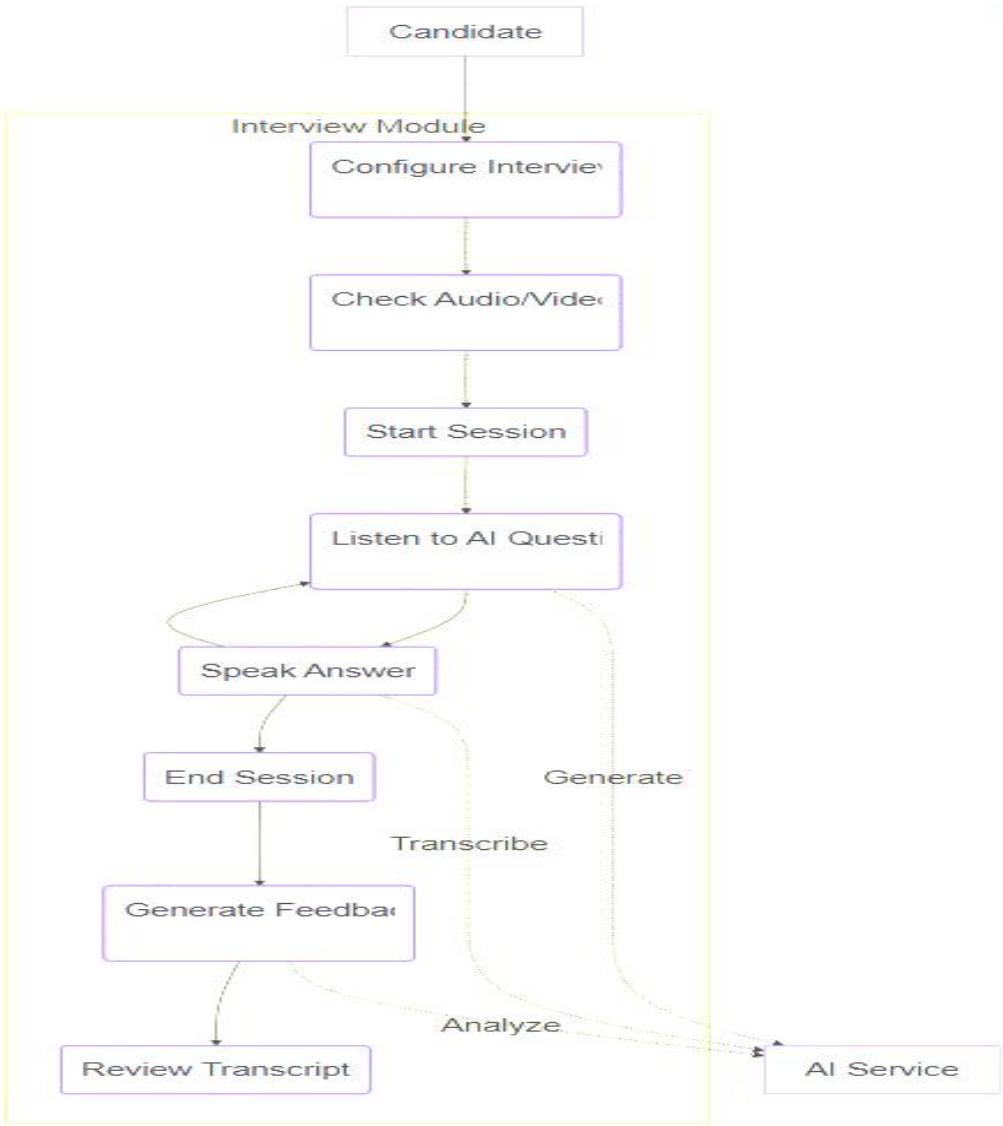
5.1.4 Sub-Module: Coding Arena Use Cases

This diagram focuses specifically on the interactions within the Coding Arena module.



5.1.5 Sub-Module: Interview Session Use Cases

This diagram details the complex interactions during an AI Mock Interview.



5.1.6 Detailed Use Case Descriptions

UC-01: Register / Login

Actor: Candidate, Administrator

Description: Allows users to create an account or access an existing one.

Preconditions: None.

Postconditions: User is authenticated and redirected to the Dashboard.

Flow:

1. User enters Email and Password.
2. System validates credentials against the Database.
3. System generates a JWT Token.

UC-04: Solve Coding Problem

Actor: Candidate

Description: The user selects a problem and attempts to solve it using the code editor.

Preconditions: User is logged in.

Postconditions: Code is saved, and a verdict (Pass/Fail) is recorded.

Flow:

1. Candidate enters the "Green Room" to check AV.
2. System connects to the AI Service.
3. AI generates a greeting based on the user's name.
4. Candidate speaks (UC9).
5. System converts Speech-to-Text.
6. AI analyzes text and generates a response.
7. System converts Text-to-Speech and plays audio.
8. Process repeats until the interview ends.

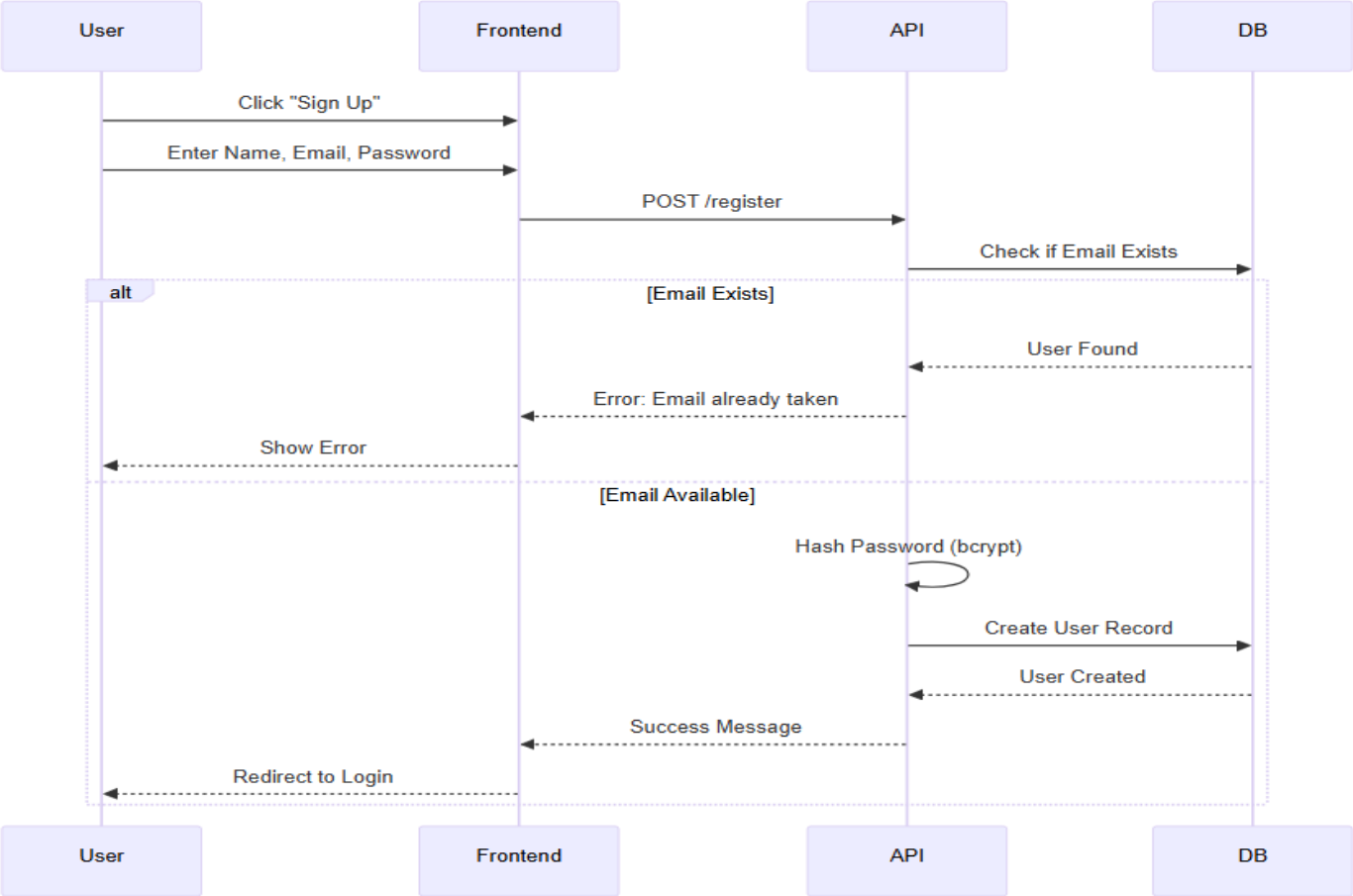
5.2 Sequence Diagrams

5.2.1 Overview

Sequence Diagrams illustrate the chronological sequence of messages exchanged between objects (or system components) to carry out a specific functionality. They provide a detailed view of the logic flow.

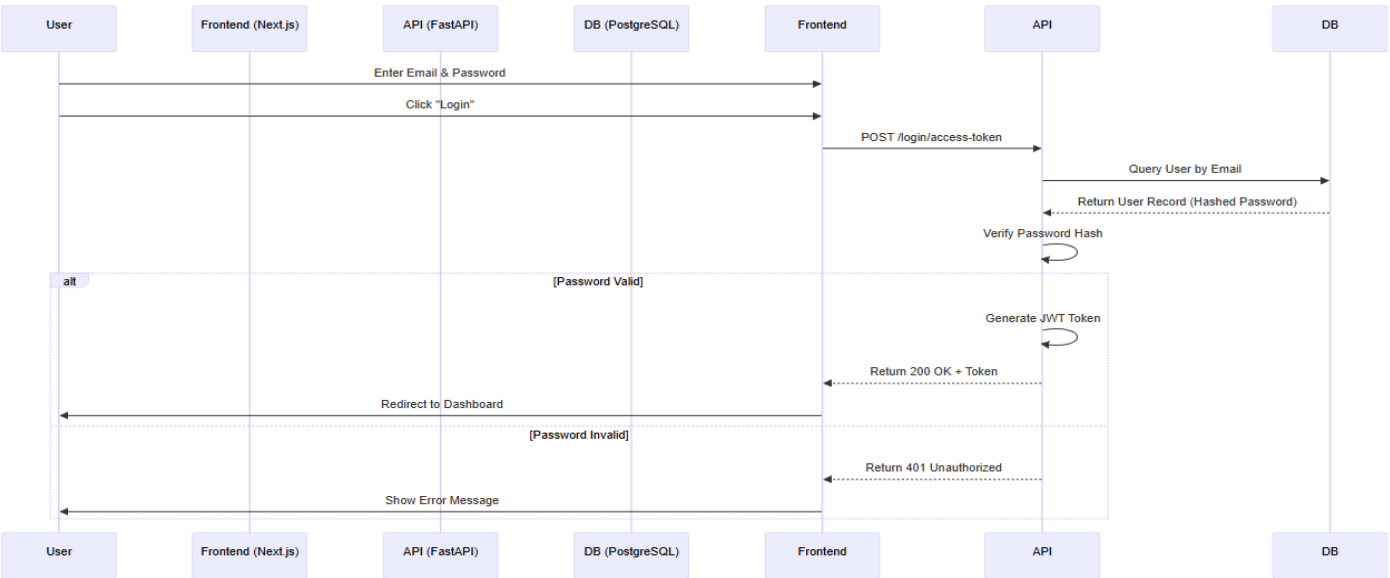
5.2.2 Sequence Diagram: User Registration

This diagram shows the flow of creating a new user account.



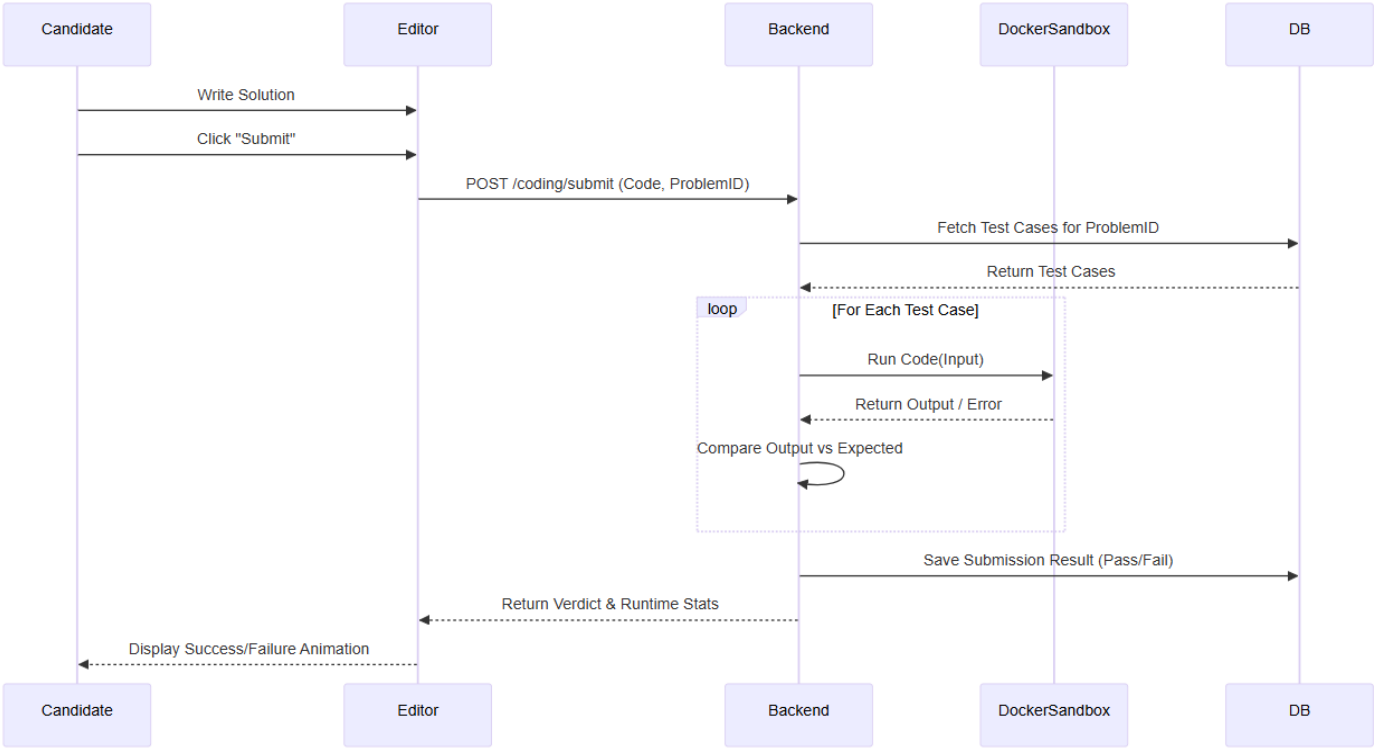
5.2.3 Sequence Diagram: User Login

This diagram details the authentication process.



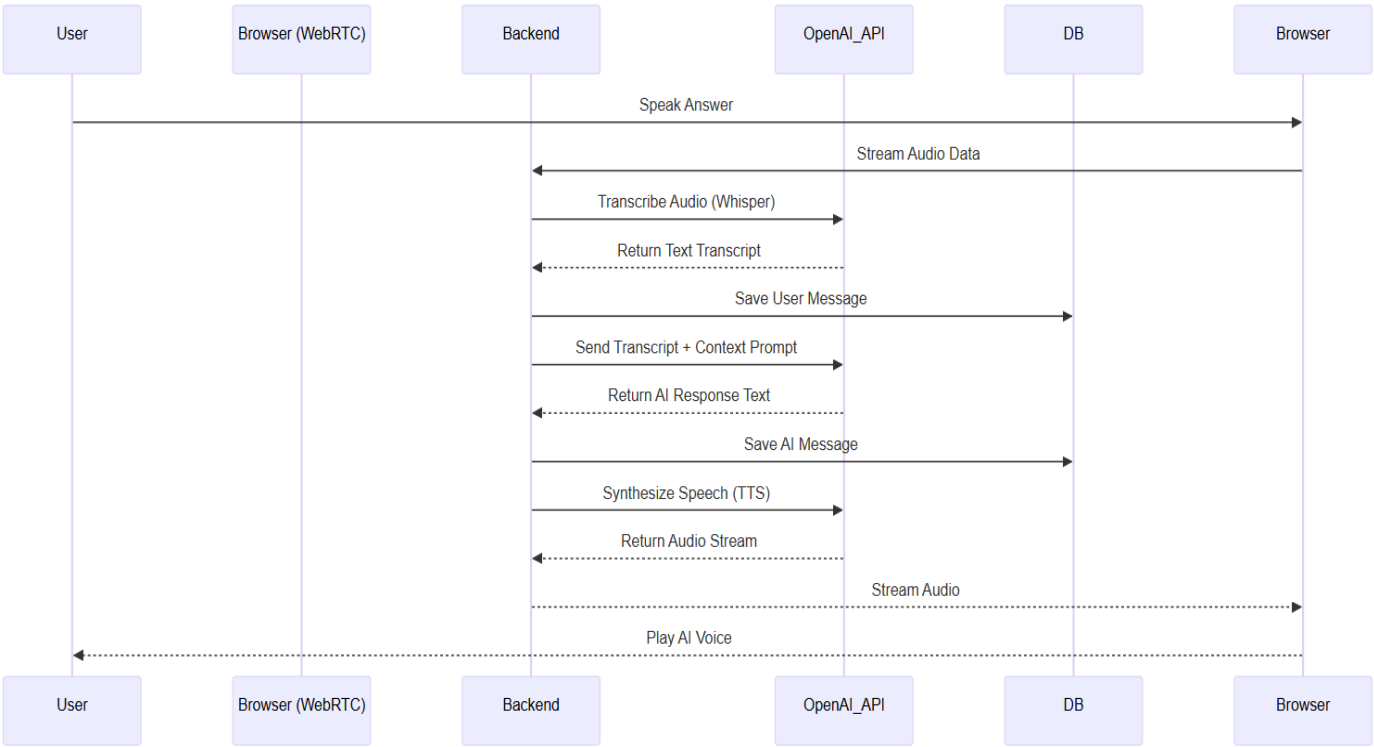
5.2.4 Sequence Diagram: Code Submission

This diagram illustrates submitting code for evaluation.



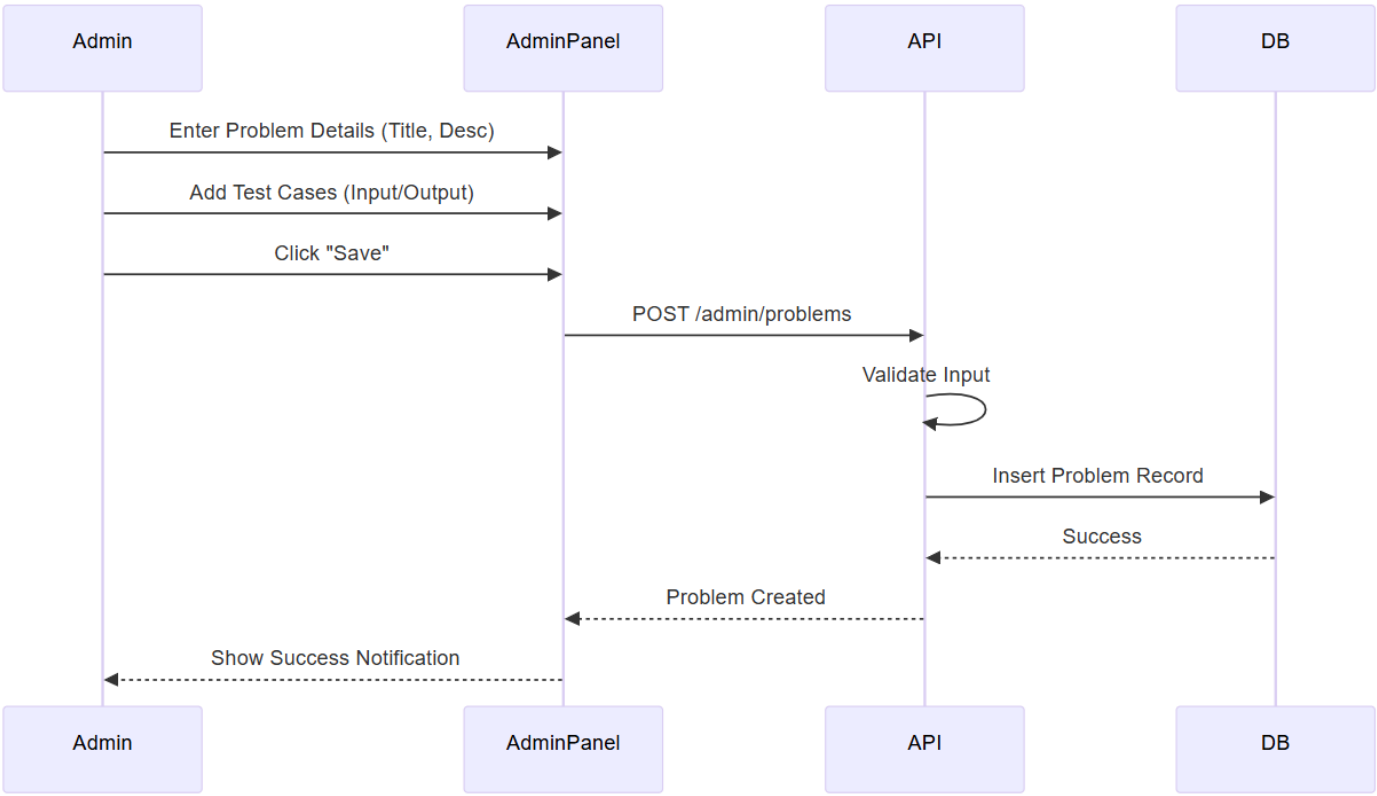
5.2.5 Sequence Diagram: AI Interview Interaction

This diagram shows the real-time data flow during an interview.



5.2.6 Sequence Diagram: Admin Adding a Problem

This diagram shows how an administrator adds a new coding challenge.



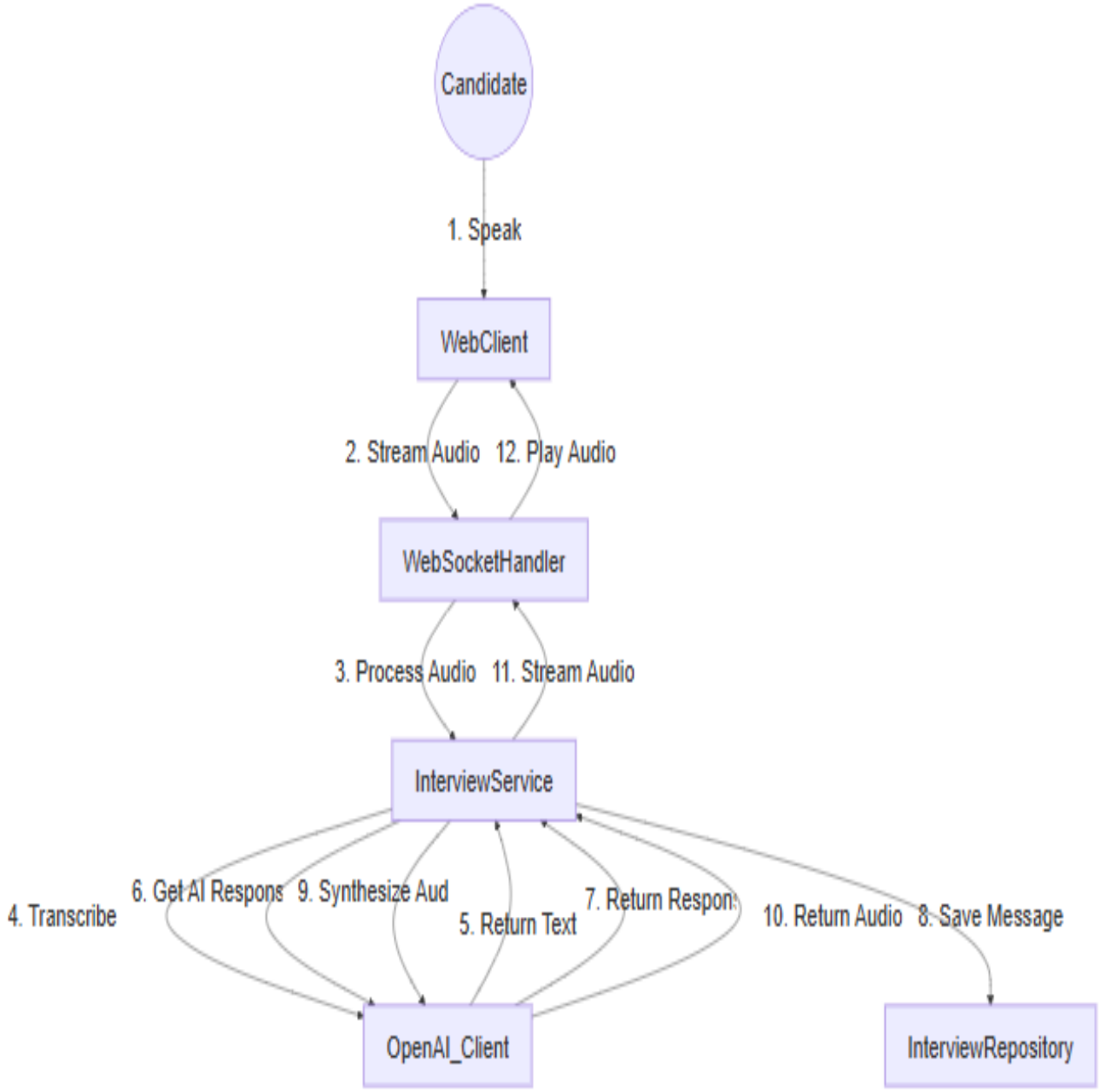
5.3 Collaboration Diagrams

5.3.1 Overview

Collaboration Diagrams (or Communication Diagrams) show the same information as sequence diagrams but focus on the structural organization of objects and their relationships rather than the timeline. They highlight who talks to whom.

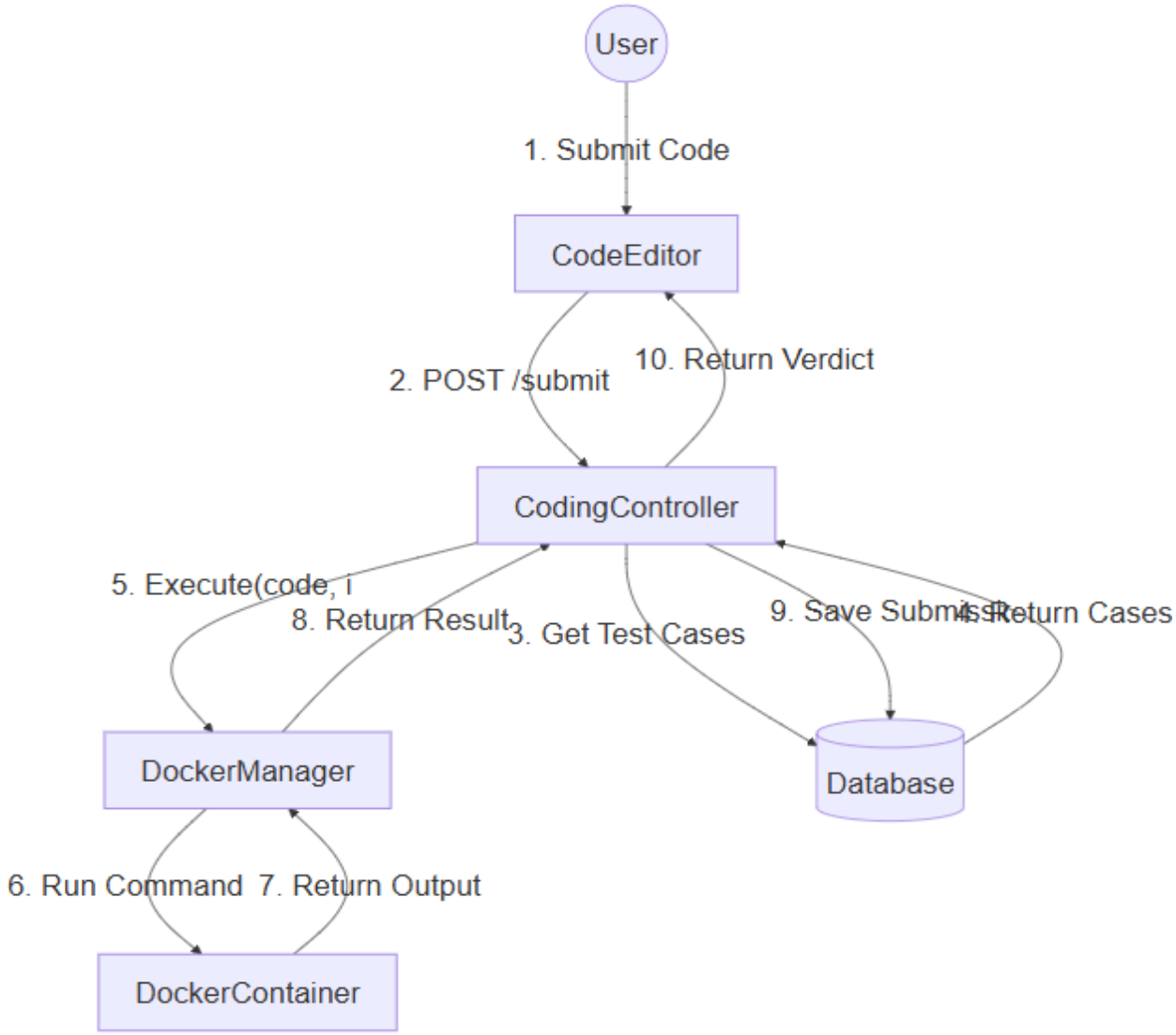
5.3.2 Collaboration: User Login Process

This diagram emphasizes the objects involved in authenticating a user.



5.3.4 Collaboration: Code Execution

This diagram shows how the system executes user code securely.



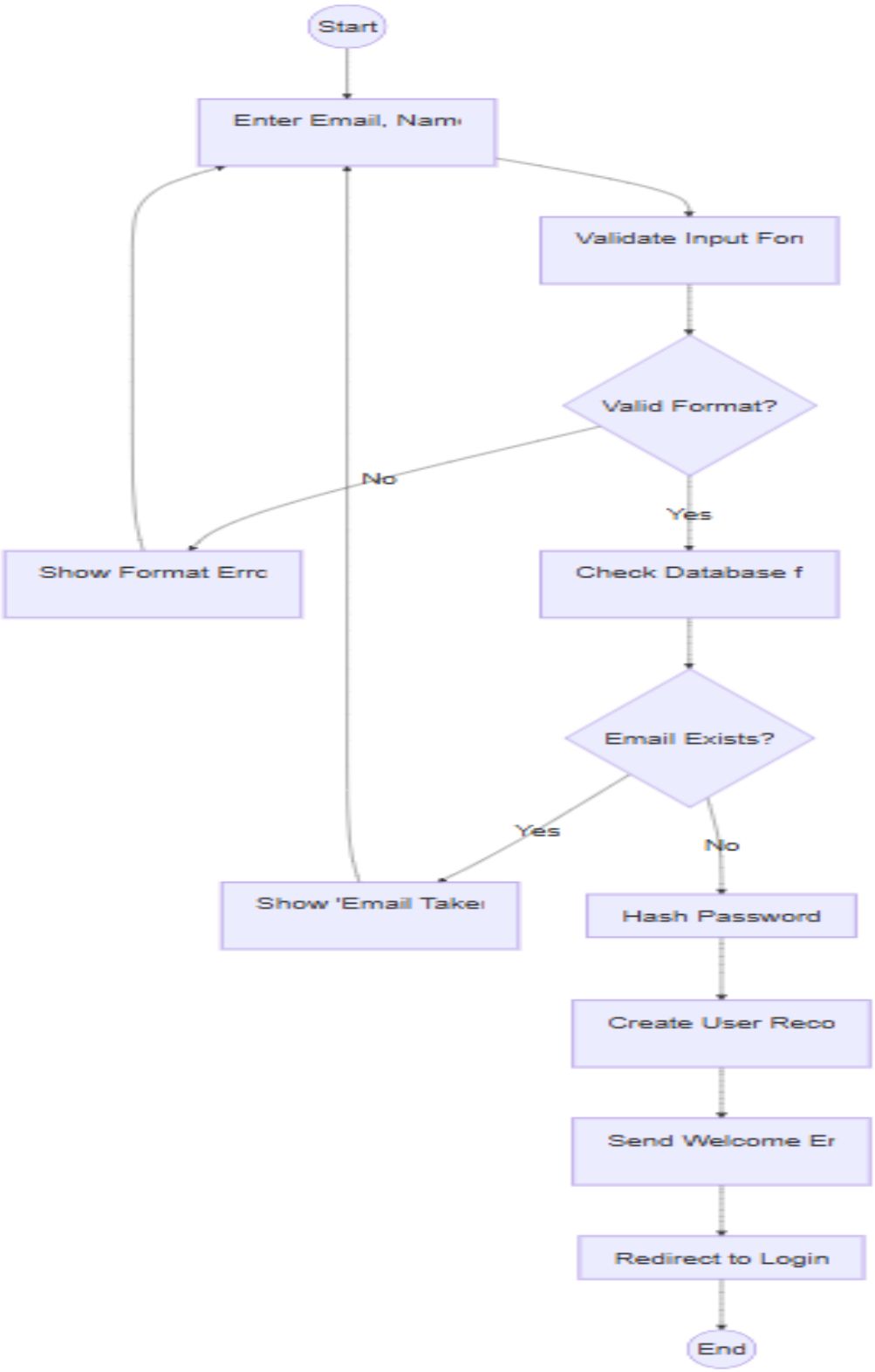
5.4 Activity Diagrams

5.4.1 Overview

Activity Diagrams model the workflow of a system and parallel processing. They are complex business logic depictions.

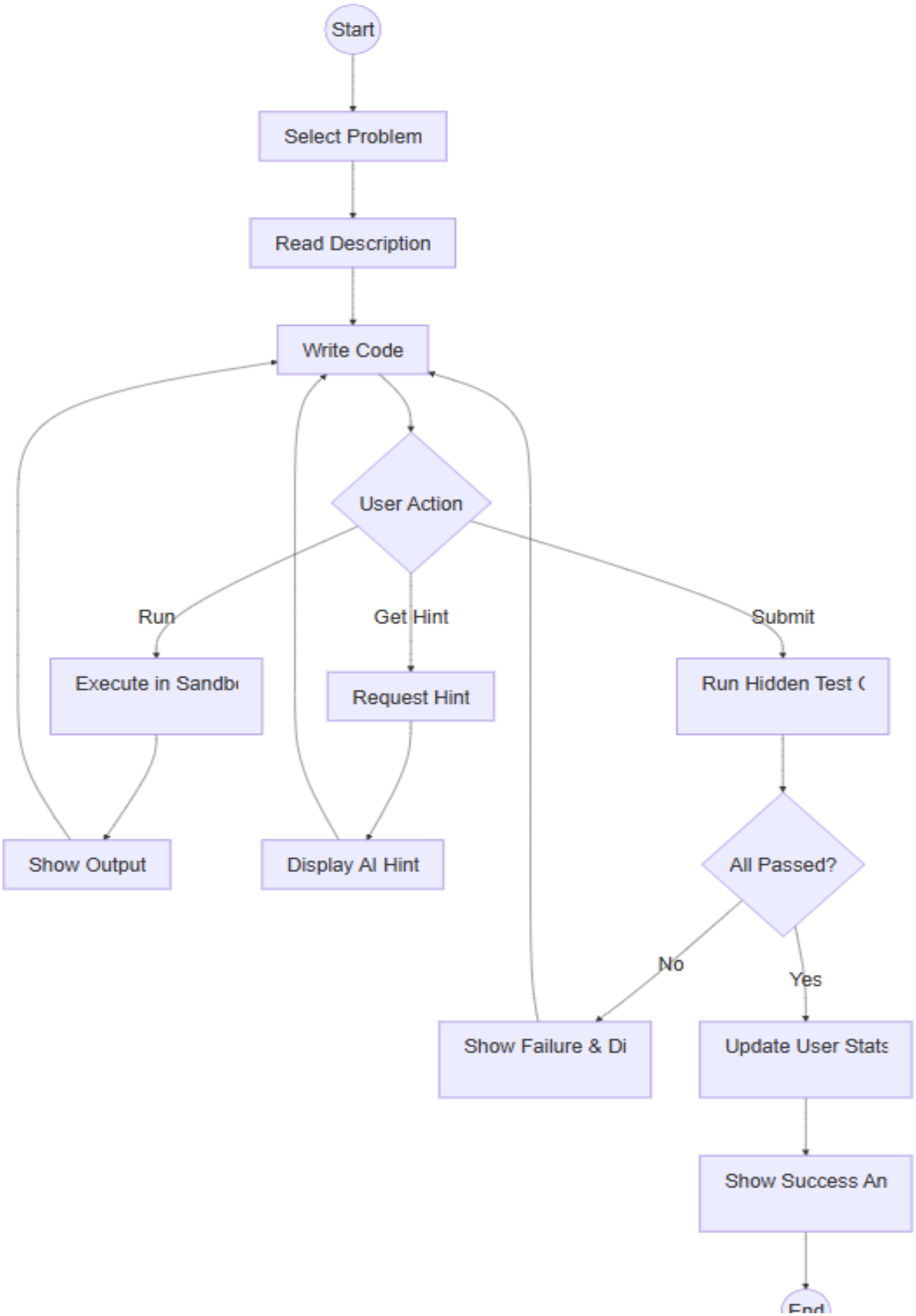
5.4.2 Activity: User Registration Flow

This diagram details the steps and decisions in the user sign-up process.



5.4.3 Activity: Solving a Coding Challenge

This diagram illustrates the user's workflow when solving a problem.

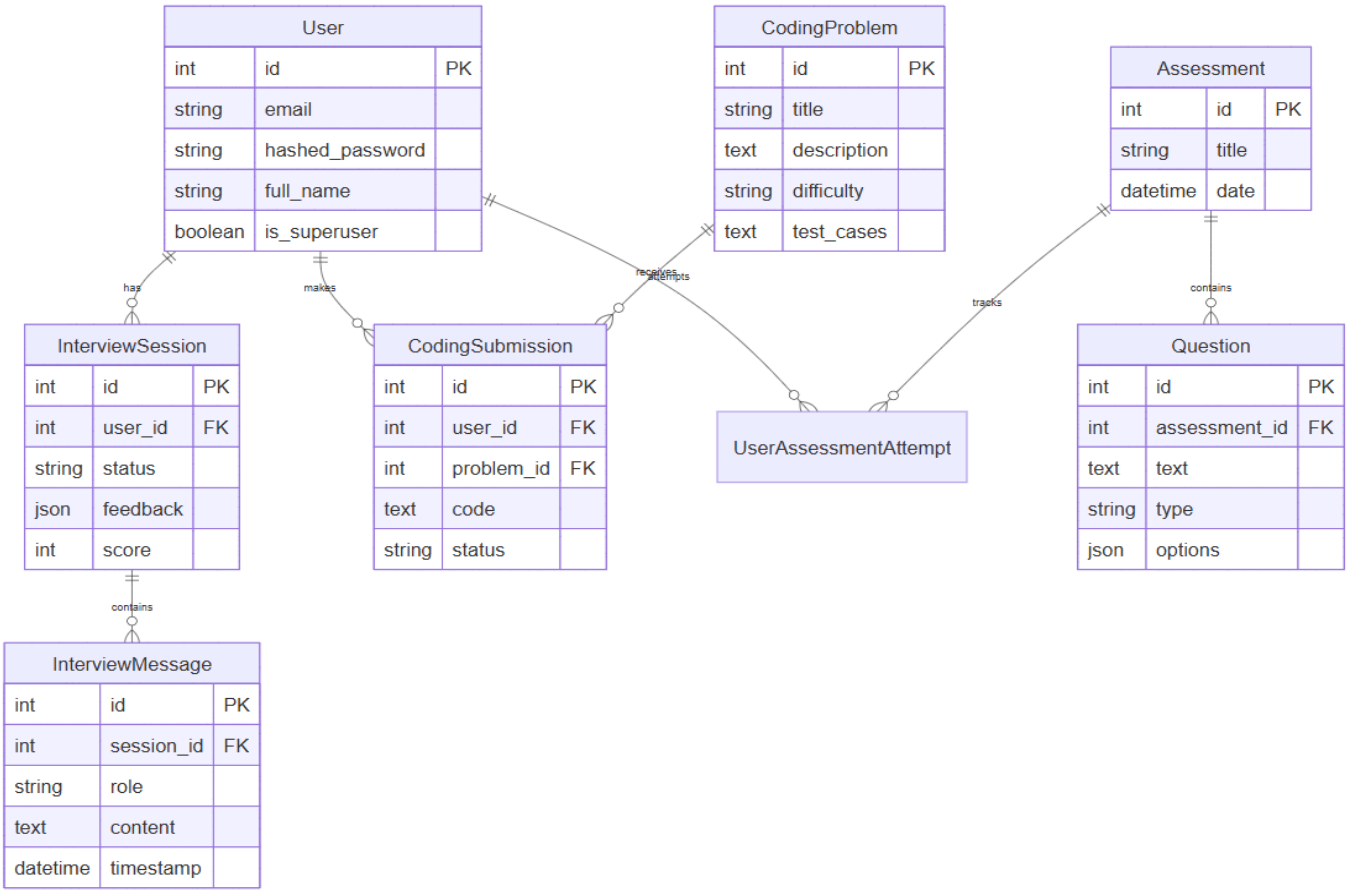


5.5 Database Design

5.5.1 Overview

The database has efficient retrieval. We use a RDBMS, specifically PostgreSQL.

5.5.2 Conceptual Schema (ER Diagram)



5.5.3 Data Dictionary

1. **Table: User**

   Stores authentication and profile information for all users.

Column Name	Data Type	Constraints	Description
Id	Integer	Primary Key	Unique key
Email	Varchar (255)	Unique, Not Null	Users email address, used for login
Hashed Password	Varchar (255)	Not Null	Bcrypt hash of the users password
Full Name	Varchar (255)	Null	Users display name on the application
Created_at	Time Stamp	Default Now()	Time Stamp of account creation.

2. Table: InterviewSession

Represents a single interview instance.

Column Name	Data Type	Constraints	Description
Id	Integer	Primary Key	Unique Identifier
User Id	Integer	Foreign Key (User. id)	The candidate taking the interview
Status	Varchar (20)	Default 'active'	Current State (active, completed, cancelled)
Feedback	JSONB	Null	Structured feedback report generated by AI
Score	Integer	Check score > 0 and score <=100	Overall performance score

3. Table: InterviewMessage

Stores the chat history (transcript) of an interview.

Column Name	Data Type	Constraints	Description
Id	Integer	Primary Key	Uniquely Identifier for the user
Session_ Id	Integer	ForeignKey (InterviewSession. id)	The session this message belongs to.
Role	Varchar (20)	Check (role IN ('user', 'assistant'))	Who is speaking (AI or the candidate)
Content	Text	Not Null	The text content of the speech
Time Stamp	Time Stamp	Default Now	When the message was spoken.

4. Table: CodingProblem

Stores the library of algorithmic challenges.

Column Name	Data Type	Constraints	Description
Id	Integer	Primary Key	Unique values
Title	Varchar (200)	Not Null	The session this message belongs to.
Description	Text	Not Null	Who is speaking (AI or the candidate)
Difficulty	Varchar (20)	CHECK (difficulty IN 'Eaasy','Medium', 'Hard'))	Problem difficulty level which is displayed
Test_ Cases	JSONB	Not Null	Array of input/output pairs for validation

5. Table: CodingSubmission

Tracks user attempts at solving problems.

Column Name	Data Type	Constraints	Description
Id	Integer	Primary Key	Unique member for the submission
User Id	Integer	Foreign Key	The user who submitted the code
Problem_Id	Integer	Foreign key CodingProblem.id	The problem being solved by user
Code	Text	Not Null	The source code is submitted
Status	Varchar(20)	Default 'Pending'	Result such as Accepted, Wrong Answer etc…

CHAPTER 06
IMPLEMENTATION

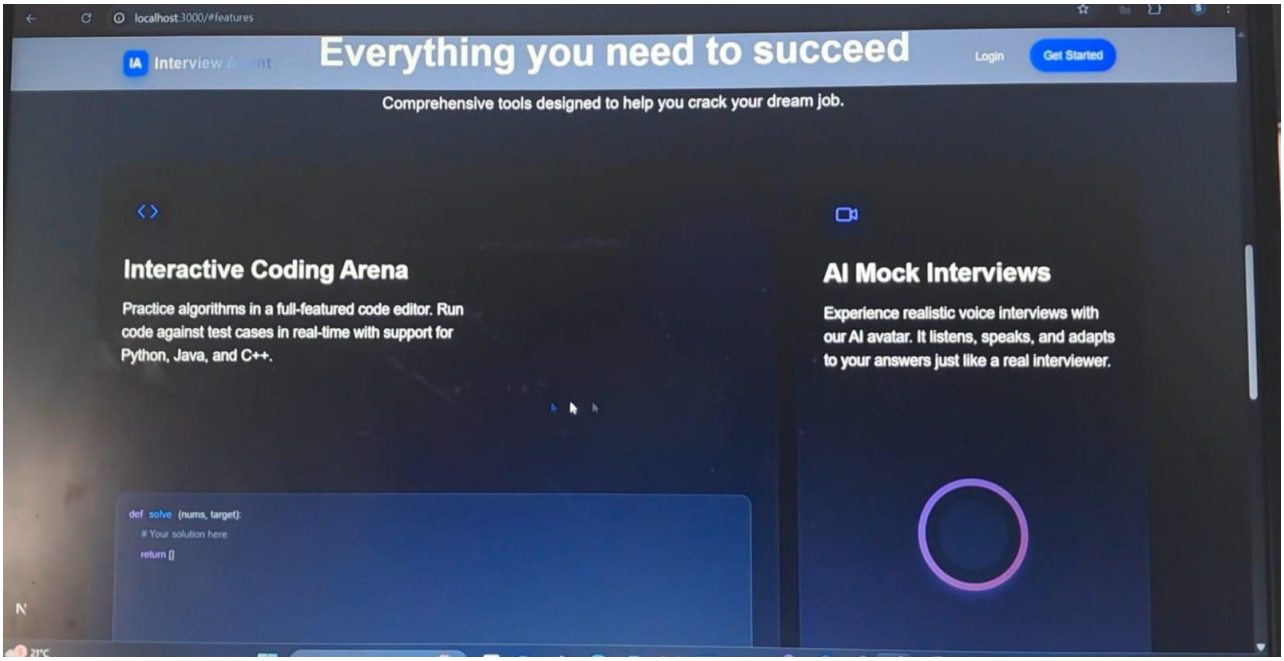
The implementation of "Interview Agent" system, verifying the successful development of all planned modules.

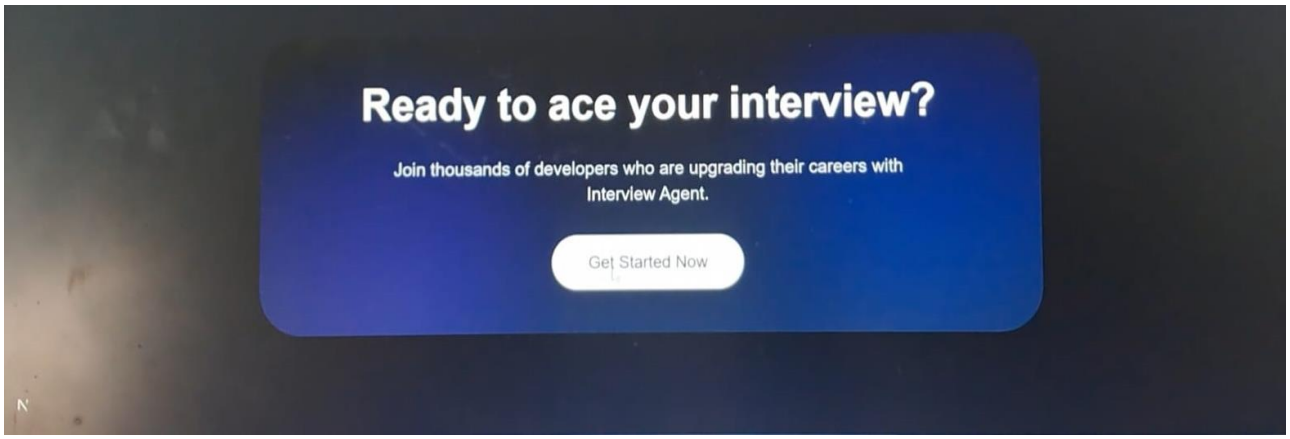
6.1 Screenshots

The following screenshots demonstrate the user flow and functional interface of the application.

6.1.1 Landing Page

Landing Page Description: The welcoming homepage of the Interview Agent application.

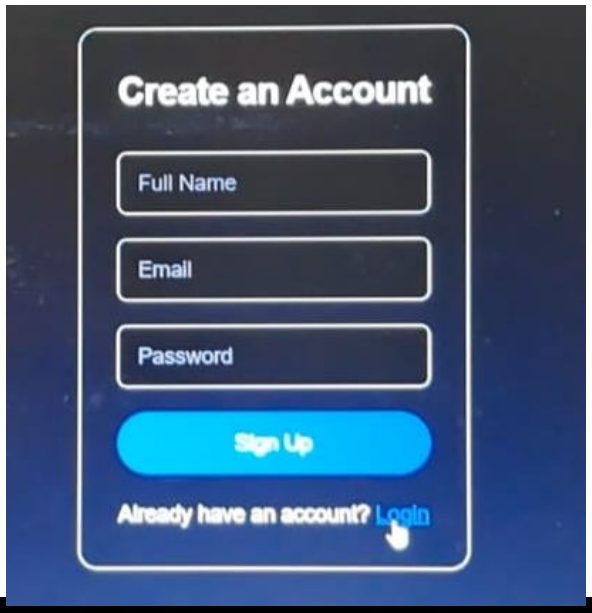




6.1.2 Authentication & User Management

Login Page, Sign Up Page

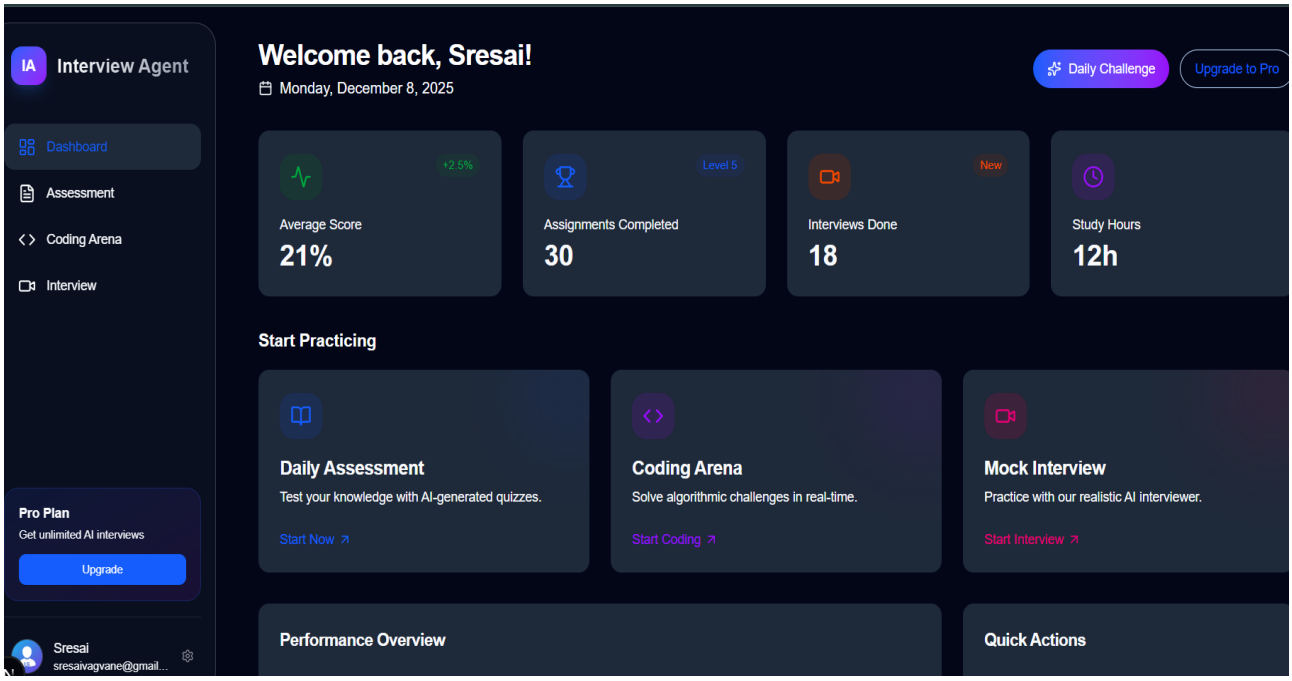
Description: To create an account & secure login for users.



6.1.3 Dashboard & Navigation

Main Dashboard

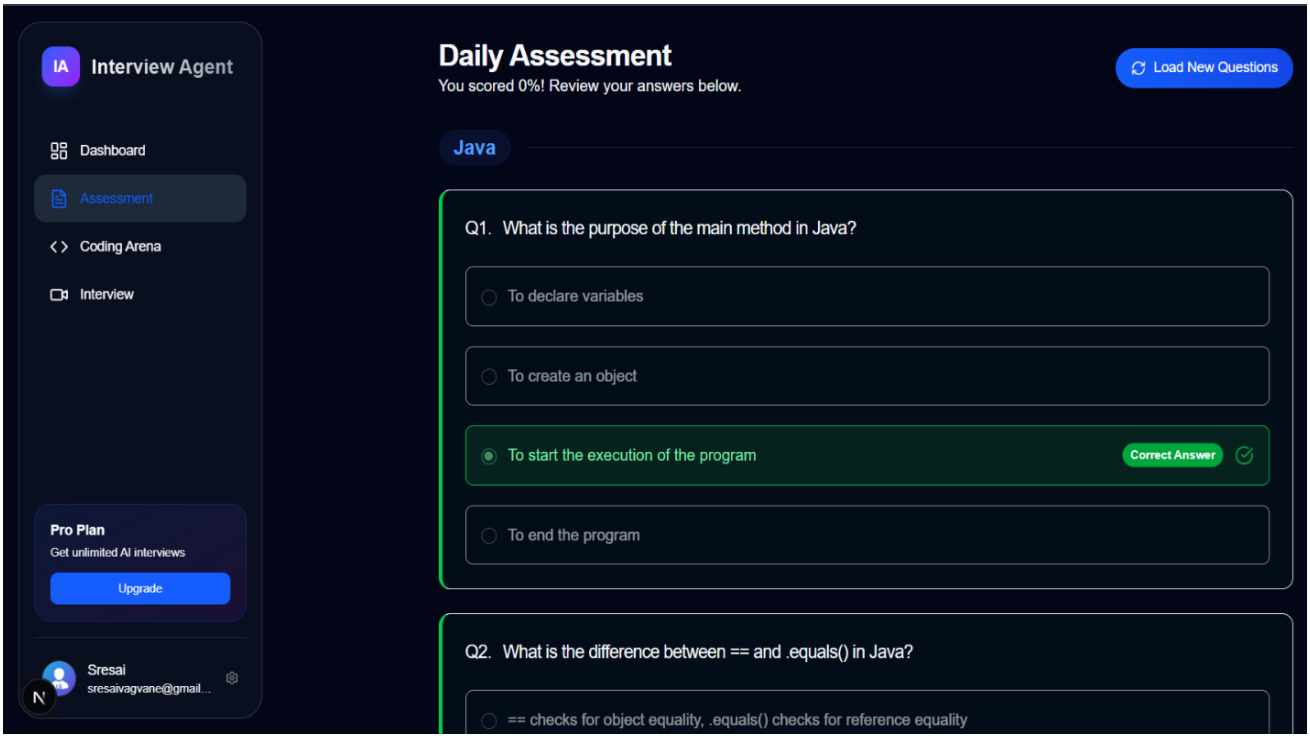
Description: The central hub showing "Daily Assessment", "Coding Arena", and "Interview Agent" modules.



6.1.4 Daily Assessment Module

Assessment Start Screen

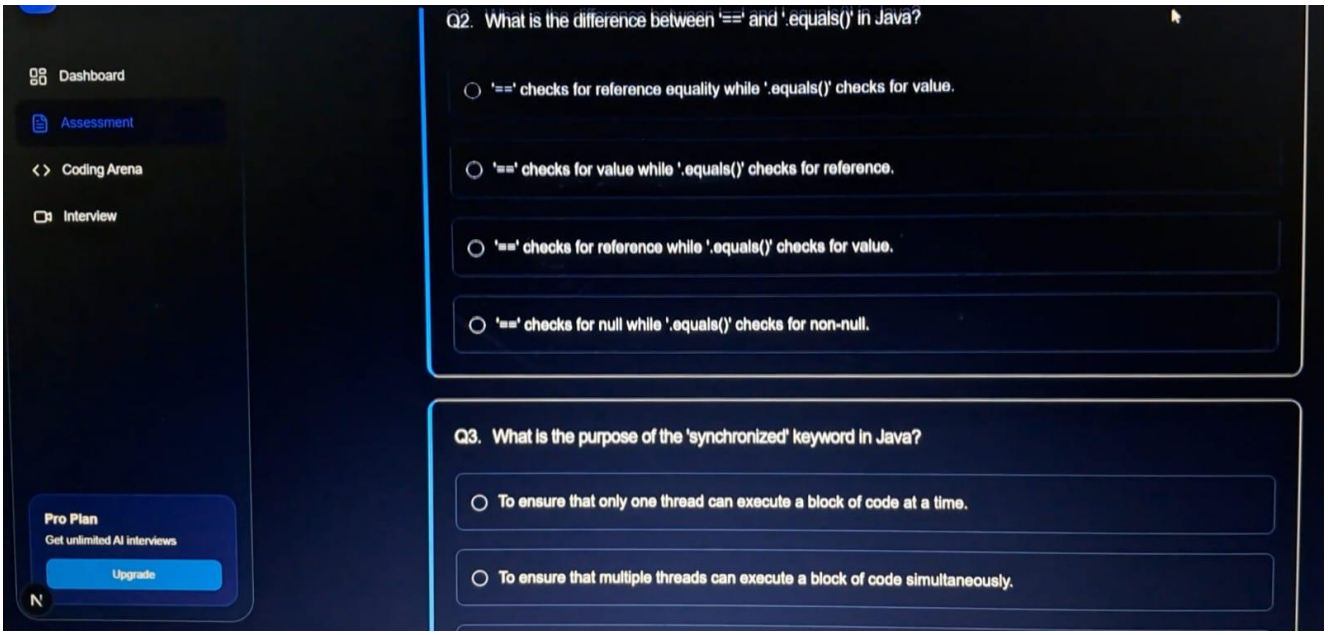
Description: Interface to begin the daily AI-generated technical quiz.



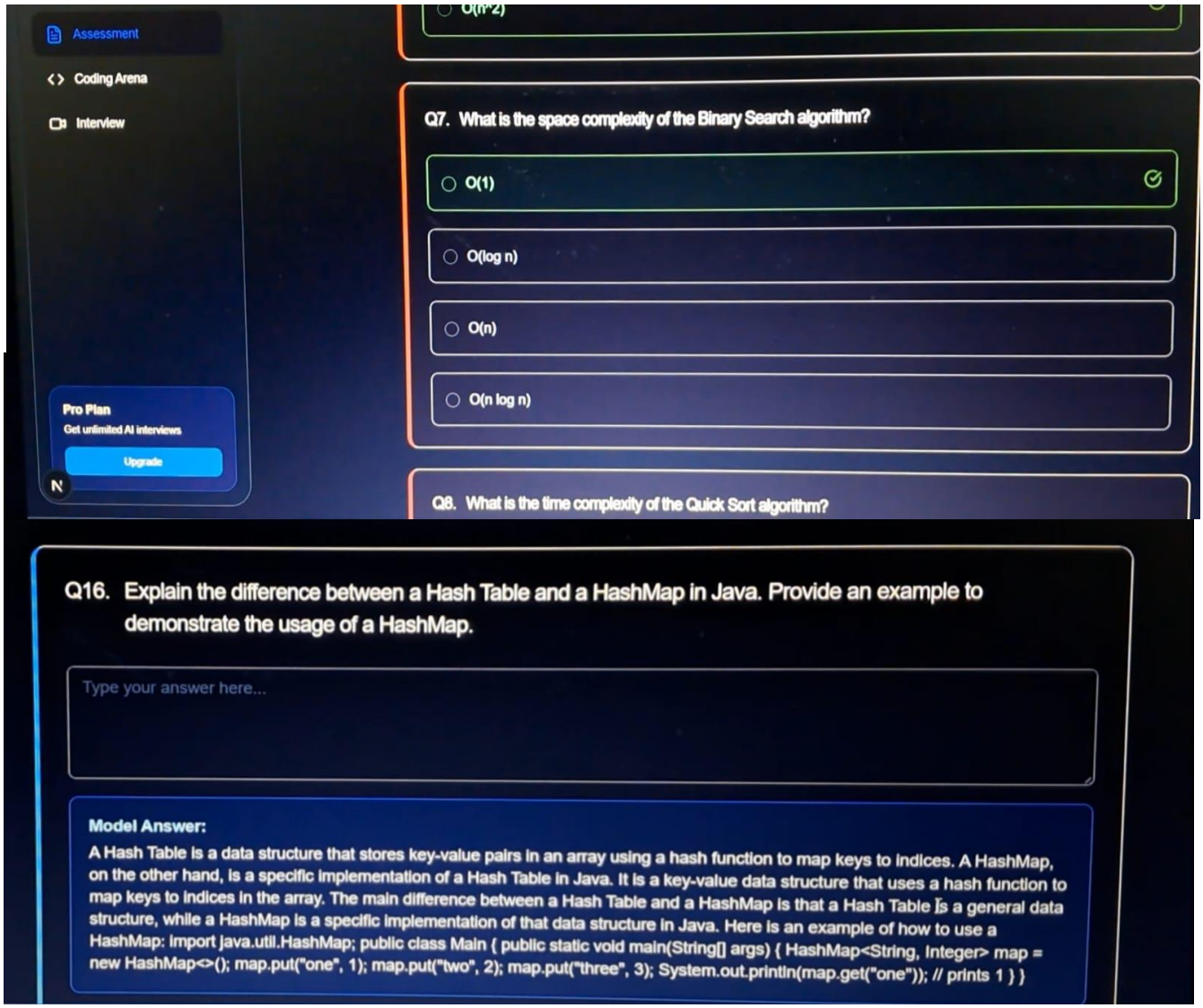
MCQ Interface

Description: AI-generated multiple-choice question (Java/DSA)

Assessment Results

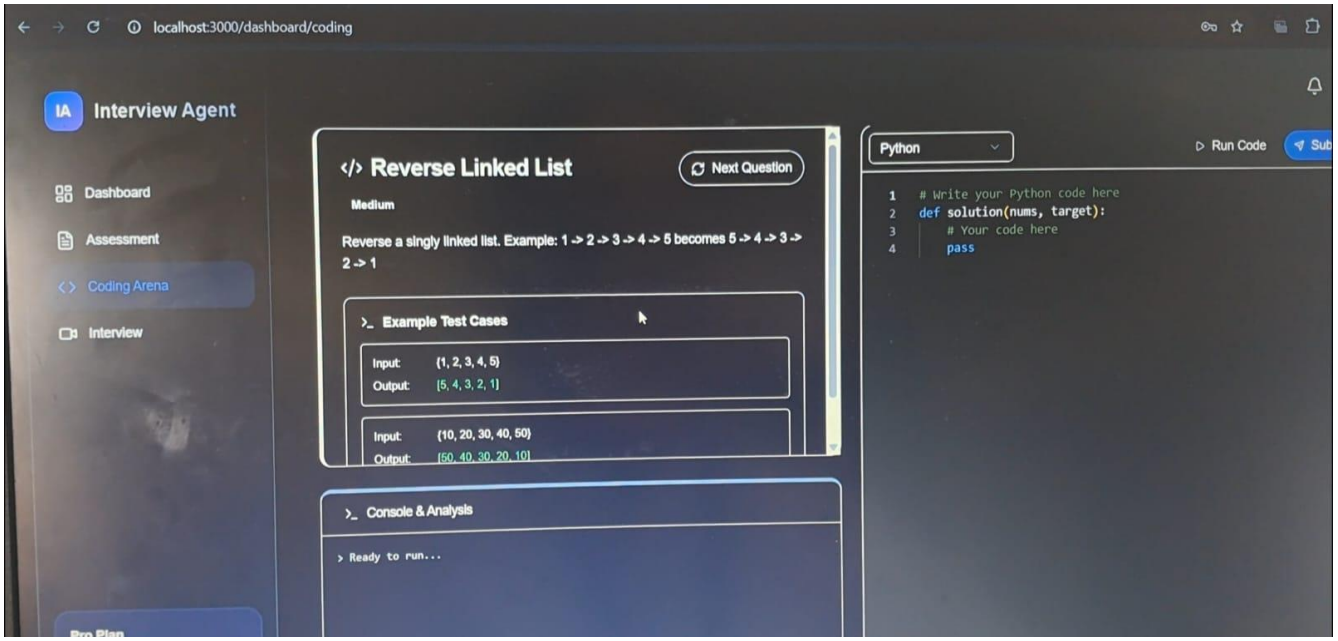


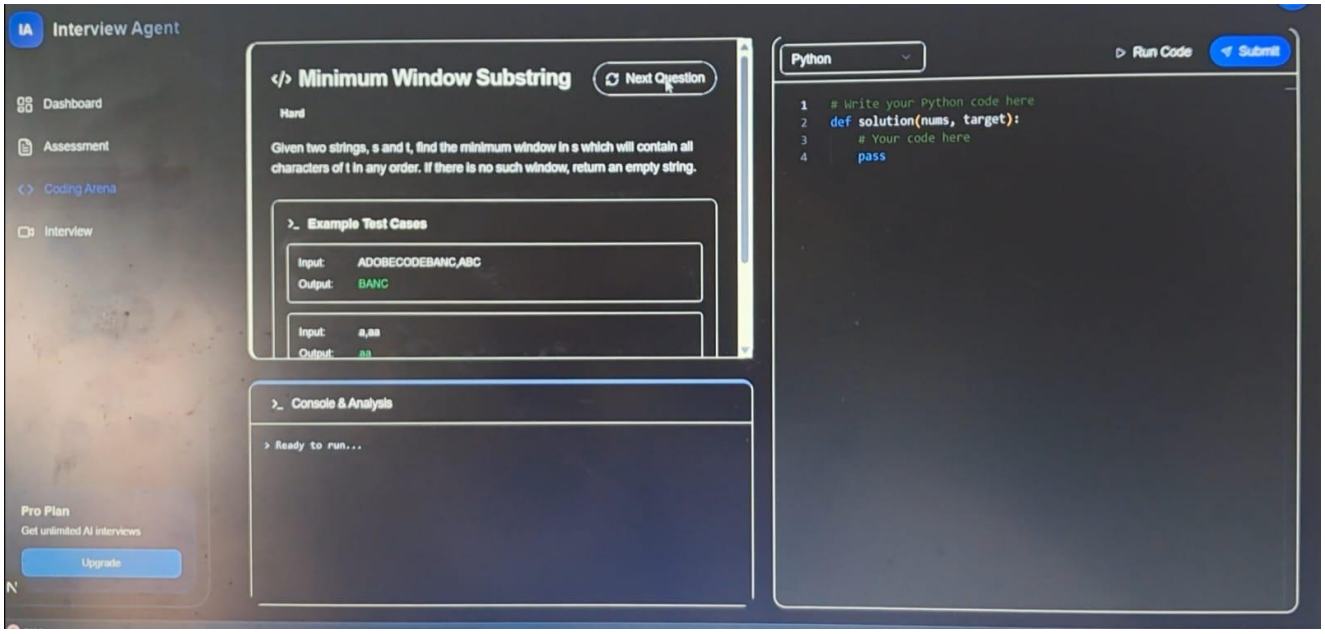
6.1.5 AI Coding Arena



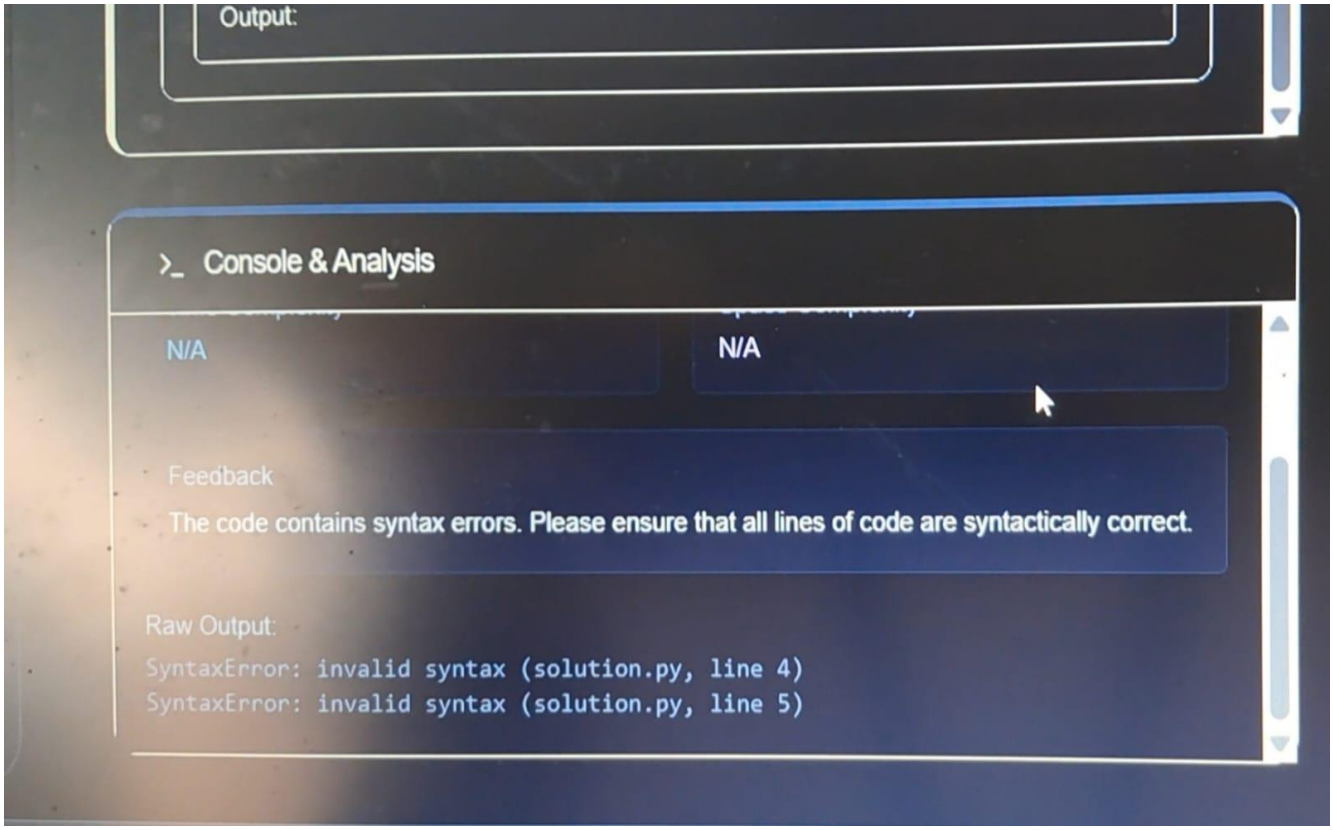
Coding Problem View

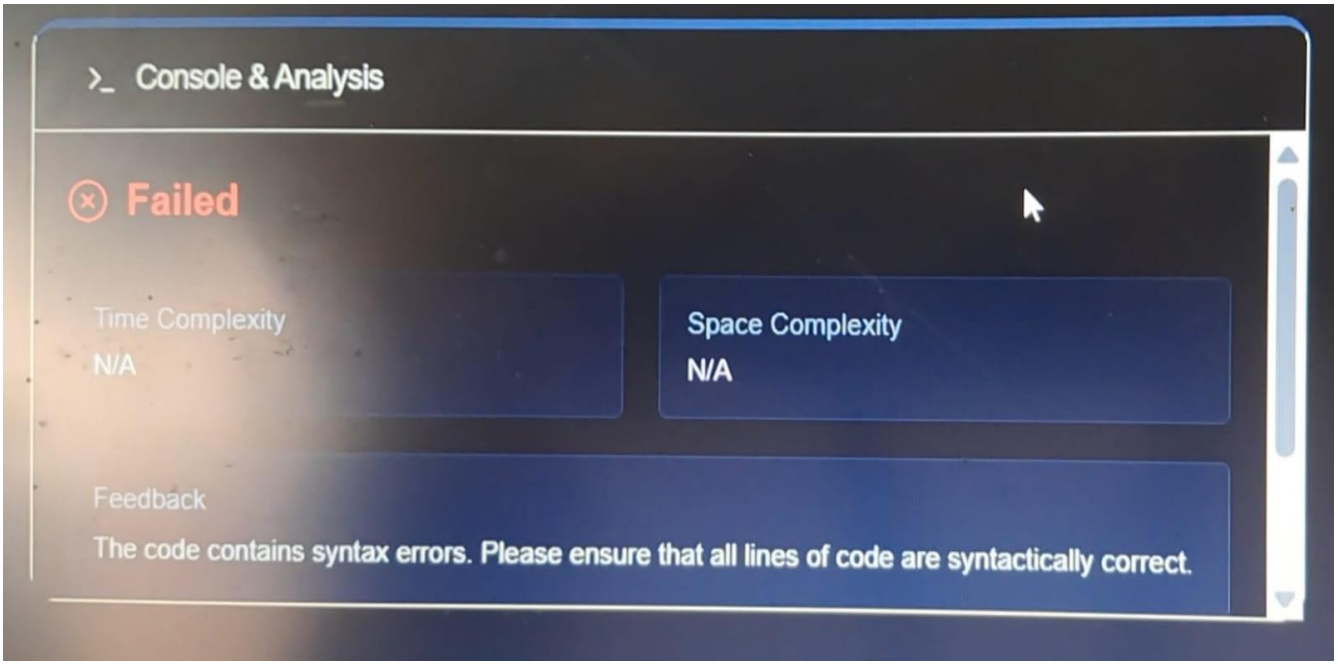
Description: The coding interface showing the problem, editor, and test case panel.





Code Execution Output
Description: Results of running the code against test cases.

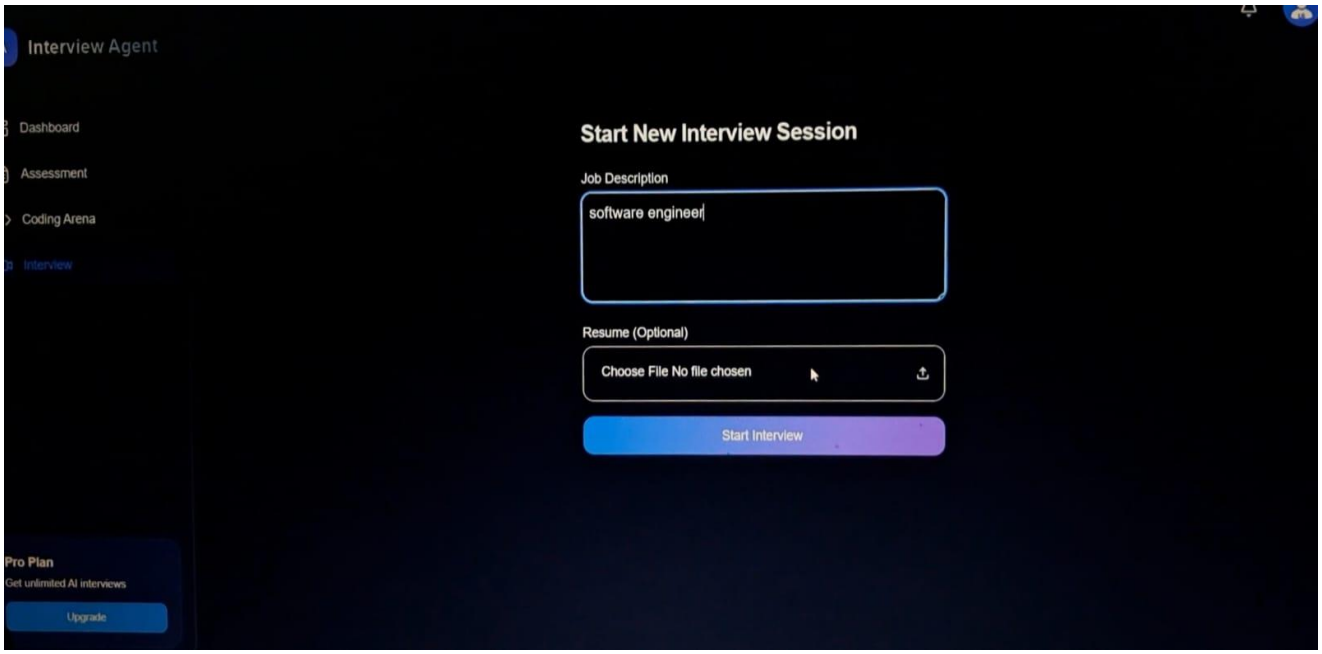




6.1.5 AI Interview Agent

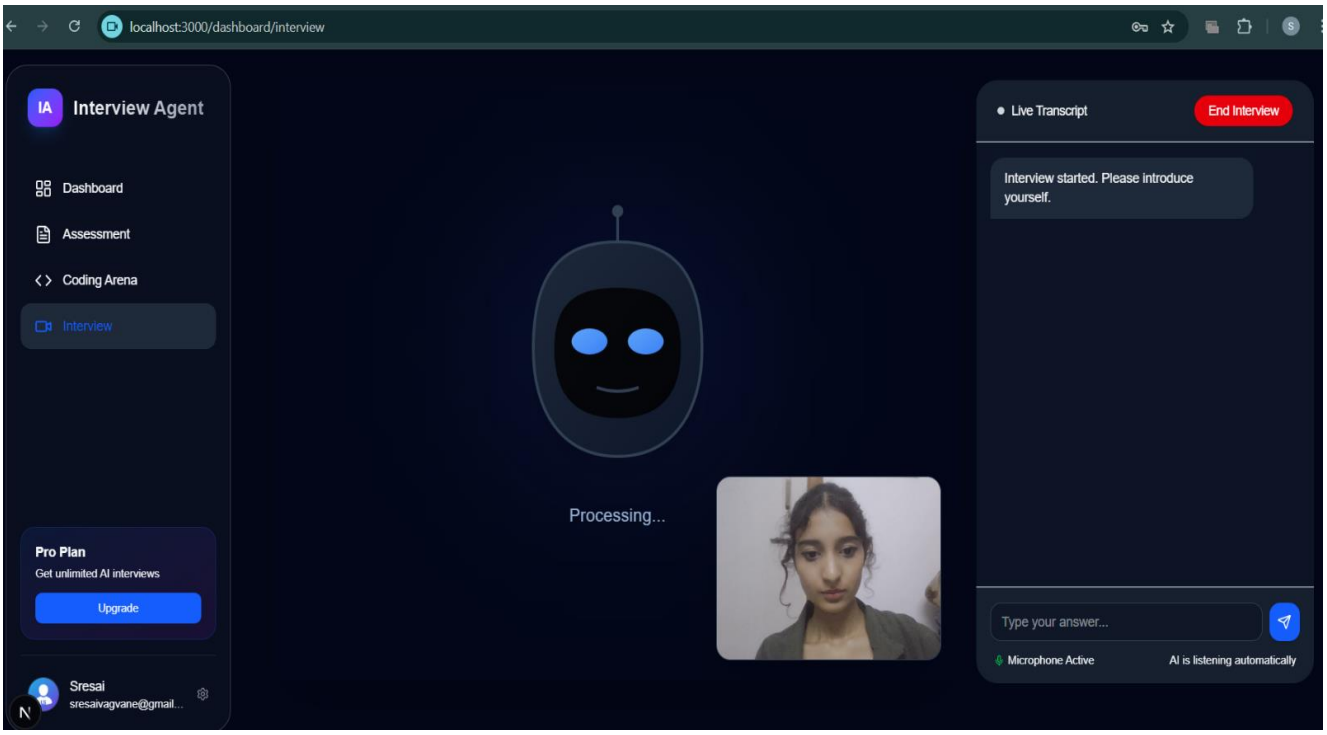
Job Description & Resume Upload

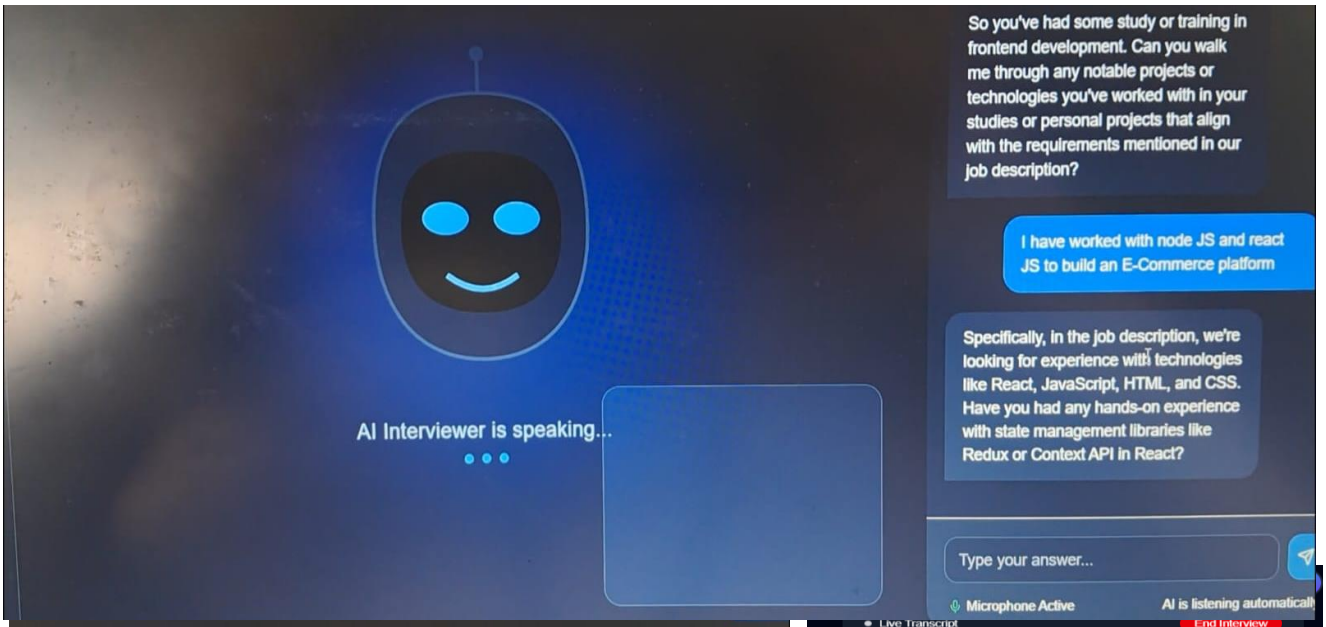
Description: Setup screen to customize the interview context.

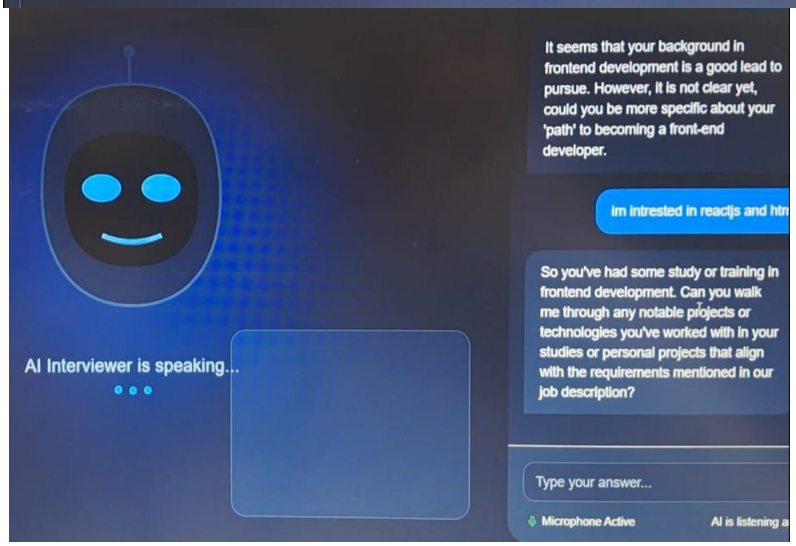


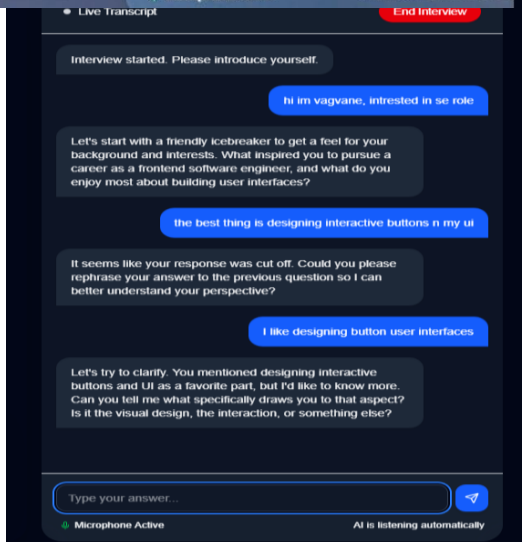
Interview Chat Interface

Description: Real-time conversation with the AI Avatar interviewer.









Feedback Report

Description: Detailed feedback generated after the interview concludes.

6.2 Key Code Snippets

This section highlights the core backend logic that powers the AI features.

6.2.1 Authentication (OAuth2) : Handles secure user login and JWT token generation

File: backend/app/api/auth.py

```
@router.post("/login/access-token", response_model=Token)
def login_access_token(
db: Session = Depends(deps.get_db), form_data: OAuth2PasswordRequestForm = Depends()
) -> Any:
user = db.query(User).filter(User.email == form_data.username).first()
if not user or not security.verify_password(form_data.password, user.hashed_password):
raise HTTPException(status_code=400, detail="Incorrect email or password")
access_token_expires = timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
return {
"access_token": security.create_access_token(
user.id, expires_delta=access_token_expires
),
"token_type": "bearer",
}
```

6.2.2 AI Assessment Generation: structured JSON questions using the LLM for the Daily Assessment

File: backend/app/services/llm.py

```
def generate_daily_questions():
prompt = """
Generate a daily technical assessment for a Full Stack Developer.
Requirements:
1. EXACTLY 5 Multiple Choice Questions (MCQs) covering "Java".
2. EXACTLY 5 Multiple Choice Questions (MCQs) covering "DSA".
3. EXACTLY 5 Multiple Choice Questions (MCQs) covering "OOPs".
4. EXACTLY 1 Subjective Question covering "Subjective".
5. RESPONSE MUST BE A SINGLE VALID JSON ARRAY.”””
response = client.chat.completions.create(
model=settings.OPENAI_MODEL_NAME,
messages=[
{"role": "system", "content": "You are a backend API that outputs strictly valid JSON array only."},
{"role": "user", "content": prompt}
],
temperature=0.4
)
return json.loads(clean_json_response(response.choices[0].message.content))
```

6.2.3 AI Coding Problem Generator: Creates unique coding challenges with test cases on demand.

File:backend/app/services/llm.py

```
def generate_coding_problem():
prompt = """
Generate a single unique coding interview problem.
{
"title": "Short Title",
"description": "Problem statement...",
"difficulty": "Easy/Medium/Hard",
"test_cases": [{"input": "...", "output": "..."}]
}
"""
response = client.chat.completions.create(
model=settings.OPENAI_MODEL_NAME, # Uses defined model (e.g., Llama-3-8B)
messages=[{"role": "user", "content": prompt}],
temperature=0.6
)
return json.loads(clean_json_response(response.choices[0].message.content))
```

6.2.4 Context-Aware Interview Logic: Manages the interview session state and generates context-aware follow-up questions.

File:backend/app/api/interview.py

```
@router.post("/{session_id}/chat", response_model=Any)
def chat_interview(session_id: int, message: Any = Body(...), db: Session = Depends(deps.get_db)):
session = db.query(InterviewSession).filter(InterviewSession.id == session_id).first()
history_msgs = db.query(InterviewMessage).filter(InterviewMessage.session_id == session.id).limit(20).all()
ai_response_text = llm.generate_interview_followup(
history=[{"role": m.role, "content": m.content} for m in history_msgs],
job_description=session.job_description,
resume_text=session.resume_text
)
ai_msg = InterviewMessage(session_id=session.id, role="assistant", content=ai_response_text)
db.add(ai_msg)
db.commit()
return {"response": ai_response_text}
```

CHAPTER 07
SOFTWARE TESTING

7.1 Introduction

Software testing is a critical phase in the development lifecycle of the "Interview Agent" system. It ensures that the application meets the functional requirements specified in the SRS, is free of critical defects, and provides a seamless user experience. This chapter details the testing strategies employed, the test environment configuration, and comprehensive test cases covering Unit, Integration, and System testing levels.

The testing process followed a "V-Model" approach, where verification and validation phases ran in parallel with development.

7.2 Testing Methodology

The following testing methodologies were adopted to ensure robust system performance:

7.2.1 Unit Testing

Individual components and functions in isolation.

Tools Used: pytest (Backend), Jest (Frontend).

Scope: Database models, utility functions (JWT generation), and algorithm helpers.

Objective: To ensure that minimal units of code perform as expected.

7.2.2 Integration Testing

Integration testing verified the interaction between different modules, specifically the communication between the FastAPI backend and the Next.js frontend, and the integration with external AI APIs (OpenAI/SambaNova).

Tools Used: Postman, Python requests (Verification Script).

Scope: API Endpoints (/login, /assessment, /interview).

Objective: To detect interface defects and data flow issues.

7.2.3 System Testing (Black-box Testing)

System testing validated the complete and integrated software product against the requirements.

Method: Manual black-box testing.

Scope: Full user flows (e.g., User Login -> Take Assessment -> View Result).

Objective: To validate end-to-end business scenarios.

7.3 Test Environment

The testing was conducted in a controlled environment to mimic production conditions.

Operating System: Windows 11

Browser: Google Chrome (v120.0), Microsoft Edge

Backend Server: Uvicorn running FastAPI (Port 8000)

Frontend Server: Next.js Dev Server (Port 3000)

Database: SQLite (Local Dev DB)

AI Provider: OpenAI API (Model: gpt-4o-mini / Meta-Llama-3.1)

7.4 Test Cases

The following sections detail the specific test cases executed.

7.4.1 Module: Authentication

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
AUTH_01	Verify User Registration with Valid Data	1. Navigate to /register. 2. Enter valid Name, Email, Password. 3. Click "Sign Up"	User account created, redirected to Login.	Account created, redirected.	PASS
AUTH_02	Verify Registration with Existing Email	1. Navigate to /register. 2. Enter an already registered Email 3. Click "Sign Up"	Error message "Email already registered".	"Email already registered" shown.	PASS
AUTH_03	Verify Login with Valid Credentials	1. Navigate to /login. 2. Enter registered Email & Password 3. Click "Login"	Token generated, redirected to Dashboard.	Dashboard loaded successfully.	PASS
AUTH_04	Verify Login with Invalid Password	1. Enter valid Email. 2. Enter wrong Password. 3. Click "Login"	Error "Incorrect email or password".	Error message displayed.	PASS
AUTH_05	Verify Logout Functionality	1. Click on User Profile Icon. 2. Select "Log out"	Session cleared, redirected to Landing Page.	Redirected to Landing Page.	PASS

7.4.2 Module: Daily Assessment (AI)

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
ASMT_01	Verify Assessment Page Load	1. Login to Dashboard. 2. Click "Start Assessment".	Assessment page loads with 16 questions.	Page loaded correctly.	PASS
ASMT_02	Verify AI Question Generation	1. Trigger generate_daily_questions(). 2. Check categories (Java, DSA, OOP).	JSON Array with 5 Java, 5 DSA, 5 OOP, 1 Subj.	Received valid JSON structure.	PASS
ASMT_03	Verify MCQ Selection	1. Click on an option for Question 1. 2. Check UI state.	Option highlights as selected.	Selection highlighted.	PASS

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
ASMT_04	Verify Assessment Submission	1. Answer all questions. 2. Click "Submit".	Score calculated, Results Modal shown.	Score: 85%, Modal displayed.	PASS
ASMT_05	Verify Result Persistence	1. Submit Assessment. 2. Go to Profile -> History.	New entry for today's assessment exists.	Entry found in History.	PASS

7.4.3 Module: Coding Arena

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
CODE_01	Verify Problem Generation	1. Navigate to Coding Arena. 2. Click "New Problem".	New problem set (Title, Desc, IO) loads.	Problem loaded.	PASS
CODE_02	Verify Code Editor Syntax	1. Type Python code in Editor. 2. Check highlighting.	Keywords highlighted (def, return).	Syntax highlighting works.	PASS
CODE_03	Verify "Run Code" Feature	1. Write print("Hello"). 2. Click "Run Code".	Output panel shows "Hello".	Output: "Hello".	PASS
CODE_04	Verify Submission Success	1. Write correct algorithm. 2. Click "Submit".	Status: "Passed", Analysis shown.	Passed, Complexity O(n).	PASS
CODE_05	Verify Submission Failure	1. Write incorrect logic. 2. Click "Submit".	Status: "Failed", Test cases failed.	Failed status shown.	PASS

7.4.4 Module: AI Interview Agent

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
INTV_01	Verify Session Initialization	1. Enter "Full Stack Dev" description. 2. Click "Start Interview".	Session created, Avatar greets user.	Chat interface loaded.	PASS
INTV_02	Verify AI Response	1. Send "Hello, I am ready.". 2. Wait for reply.	AI asks relevant follow-up question.	AI asked about experience.	PASS
INTV_03	Verify Multimedia (Mic/Cam)	1. Enable Camera permissions. 2. Speak into Mic.	Speech-to-Text converts audio.	Text appeared in input box.	PASS

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
INTV_04	Verify End Interview	1. Click "End Interview".	Session ends, Feedback report generating.	Redirected to Feedback.	PASS

7.4.5 Module: User Profile & History

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
PROF_01	Verify Activity History Load	1. Navigate to Profile. 2. Check "Activity History" tab.	Table populated with past actions.	rows displayed.	PASS
PROF_02	Verify Date Range Filter	1. Set "From Date" to yesterday. 2. Set "To Date" to today.	Table filters rows strictly within range.	Filtered correctly.	PASS
PROF_03	Verify Report Generation	1. Click "View Report" on an item.	New window opens with printable report.	Report page opened.	PASS

7.5 Integration Testing Results

Integration tests confirmed that the Frontend and Backend communicate correctly via REST APIs.

7.5.1 API Response Verification

Endpoint	Method	Payload Example	Expected Status	Actual Status
/api/v1/login/access-token	POST	{"username": "u@ex.com", "password": "123"}	200 OK	200 OK
/api/v1/assessment/daily	GET	?refresh=true	200 OK	200 OK
/api/v1/coding/submit	POST	{"code": "...", "lang": "python"}	200 OK	200 OK
/api/v1/interview/{id}/chat	POST	{"message": "Answer..."}	200 OK	200 OK

7.5.2 Database Integrity Verification

Test Scenario	Action	DB Check	Result
User Creation	Signup User	SELECT * FROM user WHERE email='...'	Record Found.
Assessment Result	Submit Quiz	SELECT * FROM user_assessment_attempt	Attempt ID generated.
Coding Problem	Generate Problem	SELECT * FROM coding_problem	Problem persisted.

7.6 Performance Testing

Basic load testing was simulated to ensure system stability under normal usage conditions.

Concurrency: Simulated 5 concurrent users accessing the Dashboard.

Result: All pages loaded within < 2 seconds.

CHAPTER 08
CONCLUSION

8.1 Conclusion

The "Interview Agent" project has been successfully planned, designed, implemented, and tested to meet the growing demand for automated technical interview preparation. The system effectively addresses the problem of limited accessibility to high-quality mock interviews by leveraging Generative AI to provide on-demand, personalized, and scalable preparation tools.

The application delivers a robust three-pillar approach to interview readiness:

1. Technical Knowledge Reinforcement: Through the "Daily Assessment" module, which uses AI to generate dynamic Java, DSA, and OOP questions, ensuring users never run out of practice material.
2. Practical Coding Proficiency: Via the "Coding Arena", which provides a real-world IDE experience with instant feedback on syntax, complexity, and correctness.
3. Communication & Behavioral Skills: Through the "AI Interview Agent", which simulates a face-to-face interview environment, offering real-time interaction and detailed feedback on candidates' performance.

Technically, the project demonstrates the successful integration of a modern Full Stack architecture (Next.js and FastAPI and OpenAI/SambaNova. The token-based authentication, reactive UI components, and resilient error handling (e.g., graceful API rate limit management) highlights the system's reliability and user-centric design.

In conclusion, the Interview Agent serves as a comprehensive, cost-effective, and always-available mentor for students and job seekers, bridging the gap between theoretical learning and industry-standard interview requirements

CHAPTER 09
FUTURE ENHANCEMENTS

The "Interview Agent" system, while currently a functional and robust application evolution. The following enhancements are proposed to extend the system's capabilities, improve user engagement, and broaden its applicability in professional training and recruitment sectors.

9.1 Real-time Collaborative Coding ("Multiplayer Mode")

Currently, the coding arena is a solitary experience. A key enhancement would be to implement a WebSocket-based collaboration layer (similar to Google Docs or VS Code Live Share).

Feature: Allow human mentors or peer interviewers to join a candidate's session.

Benefit: Facilitates real pair-programming interviews, enabling the interviewer to paste test cases or modify code snippets in real-time while observing the candidate's typing habits.

9.2 Advanced 3D Avatar Integration

The current AI interviewer implementation uses a static or simple 2D representation. Future iterations could leverage WebGL libraries (like Three.js or React-Three-Fiber) to render fully interactive 3D avatars.

Feature: Lip-syncing the avatar's mouth movements with the AI's generated audio stream.

Benefit: drastically increases immersion, simulating eye contact and non-verbal cues (nodding, facial expressions) to mimic the pressure and engagement of a face-to-face interview.

9.3 Corporate and University Dashboards

Expanding the user model to include "Organization" roles (Recruiters, Professors).

Feature: A dedicated Admin Dashboard for organizations to create custom question banks (e.g., specific to a company's tech stack) and view aggregate analytics of their candidates/students.

Benefit: Transforms the platform from a B2C study tool into a B2B Recruitment/Assessment platform.

9.4 Multi-Language Coding Support

The current MVP focuses heavily on Python and Java interaction.

Feature: Integration of a more robust compilation engine (e.g., Judge0 API) to support C++, Go, Rust, and JavaScript/TypeScript execution.

Benefit: Attracts a wider demographic of developers, including Systems Engineers (C/C++) and Frontend specialists (JS).

9.5 Gamification & Community Features

To improve user retention and motivation.

Feature: Global Leaderboards, "Streak" tracking for daily assessments, and "Badges" for mastering specific topics (e.g., "Recursion Master").

Benefit: Encourages daily practice through positive reinforcement and competitive elements.

CHAPTER 10

APPENDIX A - BIBLIOGRAPHY

References

System Core Technologies

1. Next.js Documentation. Vercel.
   Available: https://nextjs.org/docs
2. FastAPI Documentation. Sebastián Ramírez.
   Available: https://fastapi.tiangolo.com/
3. React Documentation. Meta Open Source.
   Available: https://react.dev/

AI & Machine Learning

4. OpenAI API Reference. OpenAI.
   Available: https://platform.openai.com/docs/api-reference
5. SambaNova Cloud API. SambaNova Systems.
   Available: https://cloud.sambanova.ai/

Database & ORM

6. SQLAlchemy 2.0 Documentation. SQLAlchemy Authors.
   Available: https://docs.sqlalchemy.org/en/20/
7. SQLite Documentation. SQLite Consortium.
   Available: https://www.sqlite.org/docs.html

UI/UX Libraries

8. Shadcn/UI Documentation. Shadcn.
   Available: https://ui.shadcn.com/
9. Lucide React Icons. Lucide Contributors.
   Available: https://lucide.dev/guide/packages/lucide-react
10. Tailwind CSS Documentation. Tailwind Labs.
    Available: https://tailwindcss.com/docs

Testing & Deployment

11. Pytest Documentation. Holger Krekel and pytest-dev.
    Available: https://docs.pytest.org/en/7.4.x/
12. Uvicorn Server. Encode OSS.
    Available: https://www.uvicorn.org/

APPENDIX B – USER MANUAL

1. Introduction

This manual provides step-by-step instructions for installing, configuring, and using the Interview Agent system.

2. System Requirements

Hardware: Windows/Linux/Mac PC with 8GB+ RAM.
Software:
Python 3.10+
Node.js 18.0+
Git

3. Installation Guide

Step 1: Clone the Repository
git clone https://github.com/yourusername/interview-agent.git
cd interview-agent
Step 2: Backend Setup

1. Navigate to the backend folder:
   cd backend
2. Create a virtual environment:
   python -m venv venv
   venv\Scripts\activate
3. Install dependencies:
   pip install -r requirements.txt
4. Configure Environment Variables:
   Rename .env.example to .env.
5. Run the Backend Server:
   uvicorn app.main:app --reload
   Server will start at http://localhost:8000

Step 3: Frontend Setup

1. Open a new terminal and navigate to the frontend folder:
   cd frontend
2. Install dependencies:
   npm install
3. Run the Frontend Server:
   npm run dev
   Application will be accessible at http://localhost:3000

4. User Guide

4.1 Registration & Login

1. Open http://localhost:3000 in your browser.
2. Click "Sign Up" to create a new account.
3. Enter your Name, Email, and Password.
4. Once registered, log in to access the Dashboard.

4.2 Taking the Daily Assessment

1. On the Dashboard, click "Start Assessment".
2. The AI will generate 15 MCQs and 1 Subjective question.
3. Select answers for each question.
4. Click "Submit Assessment" at the bottom.
5. View your score and detailed results instantly.

4.3 Using the Coding Arena

1. Click "Coding Arena" on the Dashboard.
2. Click "Generate New Problem" to get a fresh challenge.
3. Read the Problem Statement on the left.
4. Write your Python solution in the Code Editor on the right.
5. Click "Run Code" to test against sample cases.
6. Click "Submit" to finalize your solution (AI will check complexity and logic).

4.4 AI Interview Session

1. Click "Interview Agent" on the Dashboard.
2. Setup: Enter the Job Role/Description you are preparing for (e.g., "React Developer").
3. Optionally upload your Resume (PDF).
4. Click "Start Interview".
5. Chat: The AI Avatar will ask you questions. Type your answers or use the Microphone to speak.
6. End: Click "End Interview" to finish the session and receive a detailed feedback report.

4.5 Viewing Profile & History

1. Click your Profile Icon (top right) -> Profile.
2. View your Activity History table.
3. Use the Date Range Picker to filter past activities.
4. Click "View Report" to download a PDF summary of any session.